A Monograph-Reorganized Mathematical Companion To Fixed Sparse-Grid Quadrature Filtering And Its Analytical Gradient

Chak Wong

2026-06-10

Abstract

This note develops fixed sparse-grid quadrature filtering as a mathematically explicit high-dimensional filtering proposal for the regime where exact nonlinear filtering is unavailable but a deterministic Gaussian-surrogate value-and-gradient target is still scientifically useful. Starting from the sparse-grid quadrature nonlinear filter of Jia, Xin, and Cheng [1], the note reconstructs the Gaussian approximation and sparse-grid formulas in source order, then defines a fixed approximate likelihood scalar together with an analytical derivative of that same scalar on a declared branch. The document is written for both review and implementation: it states the exact filtering target, the narrower carried Gaussian surrogate, the fixed cloud-construction rule, the per-step value recursion, the same-scalar branch contract, the analytical gradient recursion, and the finite-difference validity rule. The central limitation is explicit. The reported scalar is not the exact nonlinear filtering likelihood, and the method does not preserve general non-Gaussian posterior shape; it is a deterministic Gaussian-surrogate lane whose usefulness depends on whether one carried Gaussian and a fixed sparse-grid moment engine are adequate for the intended regime.

1 Introduction And Scope

High-dimensional nonlinear filtering presents a familiar tension. The exact Bayesian filter propagates a full conditional law through time, but once the state transition or observation map is nonlinear that law usually no longer closes in a finite-dimensional form. In low dimension one may still tolerate a heavy deterministic quadrature rule or a richer non-Gaussian representation. In high dimension, however, the cost and structural complexity of those choices can grow quickly. This note studies one narrower but more auditable lane: a fixed sparse-grid quadrature filter that carries one Gaussian surrogate, evaluates the required nonlinear moments by a fixed deterministic sparse-grid cloud, and analytically differentiates the same declared approximate scalar on a frozen branch.

The intended reader is mathematically literate and implementation-minded. The note is written as a chapter-like companion to the monograph rather than as a survey or a software manual. Its aim is to define the exact target, the chosen surrogate object, the cloud-construction rule, the value recursion, the same-scalar derivative contract, and the verification obligations in one continuous argument. The note therefore serves two closely related roles: it is a mathematical specification of the selected deterministic Gaussian-surrogate lane, and it is an auditable handoff for later implementation and validation.

The central limitation should be fixed at the outset. FixedSGQF does not claim exact nonlinear filtering, exact nonlinear likelihood evaluation, or faithful preservation of arbitrary multimodality, skewness, or high-order global interaction structure. Its purpose is narrower. It asks whether one

carried Gaussian, together with a fixed sparse-grid moment engine and a fixed-branch value-and-gradient contract, can provide a coherent and scientifically useful approximate likelihood lane in the regime under study.

Background assumed. The reader is assumed to know multivariate Gaussian notation, state-space-model notation, Jacobian-style derivative notation, and the use of a Cholesky factor for a positive-definite covariance matrix. Sparse-grid merging, same-scalar branch identities, and fixed-cloud derivative obligations are defined in the chapters that follow.

Contents

1.1 Problem Setting And The FixedSGQF Lane

The scientific problem is high-dimensional nonlinear filtering. At time t, the exact Bayesian filter propagates and updates the full conditional law $p_\theta(x_t \mid y_{1:t})$. In nonlinear models that law is generally not available in a finite-dimensional closed form, and in high dimension it becomes unrealistic to treat exact function-valued propagation as the practical computational object. FixedSGQF therefore does not attempt to rescue exactness by notation. It selects one deterministic approximation lane whose strengths and boundaries can both be stated explicitly.

That lane is fixed sparse-grid quadrature filtering. The carried state is a single Gaussian surrogate $\mathcal{N}(m_t, P_t)$. The nonlinear moments needed by the Gaussian projection are approximated with a deterministic sparse-grid rule in standardized Gaussian coordinates. The cloud, duplicate-merge policy, covariance-factor branch, and veto rules are fixed before value and gradient evaluation so that the reported derivative differentiates the same scalar that the reported value routine computes. The gain from that design is transparency: each approximation stage is named, reproducible, and auditable. The cost is that a single Gaussian carried object cannot preserve general non-Gaussian

posterior geometry even when the quadrature rule is itself accurate for the moments it is asked to approximate.

FixedSGQF should therefore not be read as a weaker version of a richer density-approximation lane such as squared tensor-train filtering. The methods serve different goals. The present note is about the deterministic Gaussian-surrogate lane with a fixed cloud and a same-scalar analytical gradient.

1.2 The Selected Approximation Lane In One Page

The lane studied here is defined by three deliberate approximations.

1. **Gaussian carried state.** Instead of carrying the full filtering law, the method carries one Gaussian summary $\mathcal{N}(m_t, P_t)$.
2. **Sparse-grid moment closure.** The nonlinear Gaussian moments that define the prediction and observation projection are approximated by a fixed sparse-grid rule in standardized coordinates.
3. **Fixed-branch differentiation.** The reported gradient is the derivative of a declared deterministic surrogate scalar on a frozen branch, not the derivative of a live adaptive algorithm.

The resulting accumulated scalar is

$$\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta) = \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{FSGQ}}(\theta), \tag{1.1}$$

where each increment is the Gaussian innovation log likelihood implied by the sparse-grid moment approximation. The exact object, the carried surrogate, and the accumulated scalar are therefore different in kind:

$$\begin{aligned} \text{exact object:} \quad & p_\theta(x_t \mid y_{1:t}), \\ \text{carried surrogate:} \quad & \mathcal{N}(m_t, P_t), \\ \text{declared deterministic scalar:} \quad & \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta). \end{aligned} \tag{1.2}$$

This distinction is the governing contract of the note. Later sections unpack one rung of that contract at a time rather than reintroducing the full thesis from scratch.

1.3 Approximation Hierarchy And Coordinate Walk

The method is easiest to read in the same order in which the computation is assembled. Start with the exact Bayesian recursion. Replace the full filtering law by one Gaussian surrogate. Express the needed nonlinear Gaussian moments in standardized coordinates. Approximate those moments by a sparse-grid rule. Finally, freeze the branch so that the value and the derivative refer to the same declared scalar.

The main coordinates are:

$$\xi \in \mathbb{R}^{n_x}, \qquad \xi \sim \mathcal{N}(0, I_{n_x}) \qquad \text{standard quadrature coordinate,} \tag{1.3}$$

$$x = m + C\xi, \quad CC^\top = P \qquad \text{physical state coordinate under a carried Gaussian,} \tag{1.4}$$

$$a = f_\theta(x), \qquad \text{transition image used for prediction,} \tag{1.5}$$

$$z = h_\theta(x), \qquad \text{observation image used for the innovation.} \tag{1.6}$$

The sparse grid is built once in ξ-space and then placed into physical state space through the current Gaussian factor. This is why one saved cloud can be reused across time while the physical points move with m_t, P_t, and θ.

1.4 Contribution Summary And Reading Guide

The chapter now unfolds in four large movements. First, the formal filtering and Gaussian-projection foundation is fixed. Second, the low-dimensional construction of the sparse-grid rule is developed from scalar and tensor-product examples through merged clouds and the UKF bridge. Third, the fixed cloud is inserted into the filtering value recursion and then into the same-scalar gradient recursion. Fourth, the resulting value-and-gradient lane is verified, positioned against neighboring methods, and bounded by explicit non-claims.

Implementation workers should treat the equations, contracts, and worked examples as the authoritative coding material. Readers concerned mainly with the sparse grid itself should move first to the low-dimensional construction chapter and then return to the formal recursion once the cloud picture is clear.

2 Exact Filtering And Gaussian Projection

Use the additive-noise state-space model

$$X_t = f_\theta(X_{t-1}) + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_\theta), \tag{2.1}$$

$$Y_t = h_\theta(X_t) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_\theta), \tag{2.2}$$

with $X_t \in \mathbb{R}^{n_x}$, $Y_t \in \mathbb{R}^{n_y}$, and parameter $\theta \in \mathbb{R}^p$. Throughout this note, the process noises η_t are assumed zero mean, mutually independent of the observation noises ε_s for all relevant time indices, independent across time within the process-noise family, and independent of the past latent states and observations conditional on the model. Likewise, the observation noises ε_t are assumed zero mean, independent across time within the observation-noise family, and independent of the latent states conditional on the declared additive observation model. These assumptions are exactly what let the filtering factorization and the later covariance identities use Q_θ and R_θ as the declared additive noise covariances. The exact Bayesian prediction and update are

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) \, \mathrm{d}x_{t-1}, \tag{2.3}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{\int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}) \, \mathrm{d}x_t}. \tag{2.4}$$

These are the conceptual filtering equations used by Gaussian approximation filters in Jia, Xin, and Cheng [1, Section 2]. In nonlinear models, the integrals in (2.3) and (2.4) usually do not close in finite-dimensional form. Fixed SGQF replaces those exact densities by a Gaussian moment projection.

2.1 Report-Wide Object Map And Implementation Conventions

The report uses the following principal objects throughout the value and gradient paths. An implementation worker should treat this table as the core object inventory for the note.

symbol	meaning	shape
$\xi^{(r)}$	standardized sparse-grid node in $\mathbb{R}^{n_x}$	n_x
w_r	accumulated signed weight after duplicate merge	scalar
m_t, P_t	carried Gaussian mean and covariance after update	n_x, $n_x \times n_x$
m_t^-, P_t^-	predictive Gaussian mean and covariance before update	n_x, $n_x \times n_x$
C_t, C_t^-	covariance factors with $P_t = C_t C_t^\top$, $P_t^- = C_t^-(C_t^-)^\top$	$n_x \times n_x$
$\chi_{t-1}^{(r)}$	previous-state physical cloud point	n_x
$a_t^{(r)}$	transition image of $\chi_{t-1}^{(r)}$	n_x
$\chi_t^{(r)}$	predictive physical cloud point	n_x
$z_t^{(r)}$	observation image of $\chi_t^{(r)}$	n_y
$\bar{z}_t$	predicted observation mean	n_y
S_t	innovation covariance	$n_y \times n_y$
$C_{xz,t}$	state-observation cross-covariance	$n_x \times n_y$
v_t	innovation residual $y_t - \bar{z}_t$	n_y
K_t	Gaussian projection gain	$n_x \times n_y$
$\widehat{\ell}_t^{\mathrm{FSGQ}}$	one-step deterministic approximate log-likelihood increment	scalar

Three implementation conventions are fixed report-wide.

1. **Factor convention.** Every covariance factor is the lower triangular Cholesky factor with positive diagonal, written $C(P) = \mathrm{chol}(P)$, whenever the declared branch is valid.

2. **Solve convention.** Formulas may use S_t^{-1} for compact mathematics, but the implementation contract is to compute all such actions by linear solves against the declared Cholesky factor of S_t, not by forming an explicit inverse.

3. **Noise convention.** Additive process noise enters analytically through the covariance term Q_θ in predictive moments; additive observation noise enters analytically through R_θ in the innovation covariance. This report does not use state augmentation for these additive noises.

When floating-point accumulation produces a covariance that is not exactly symmetric, the report assumes explicit symmetrization by $(P + P^\top)/2$ before branch checking. If the resulting matrix is not positive definite on the declared Cholesky branch, the branch is vetoed rather than silently repaired. Any jitter, clipping, or eigenvalue flooring policy would define a different scalar than the one studied here.

2.1.1 Gaussian Projection From Moments

Assume the predictive state is represented by

$$X_t^- \sim \mathcal{N}(m_t^-, P_t^-). \tag{2.5}$$

Let

$$Z_t = h_\theta(X_t^-) \tag{2.6}$$

be the noiseless predicted observation. The Gaussian projection needs

$$\bar{z}_t = \mathbb{E}[Z_t], \tag{2.7}$$

$$S_t = R_\theta + \mathbb{E}[(Z_t - \bar{z}_t)(Z_t - \bar{z}_t)^\top], \tag{2.8}$$
$$C_{xz,t} = \mathbb{E}[(X_t^- - m_t^-)(Z_t - \bar{z}_t)^\top]. \tag{2.9}$$

Given these moments, define

$$v_t = y_t - \bar{z}_t, \qquad K_t = C_{xz,t} S_t^{-1}, \tag{2.10}$$
$$m_t = m_t^- + K_t v_t, \qquad P_t = P_t^- - K_t S_t K_t^\top. \tag{2.11}$$

Proposition 1 (Affine projection)**.** *Assume the innovation covariance S_t is square and invertible, so the solve or inverse notation below is well defined, and all matrix products are conformable under the stated object map. Among estimators of X_t^- of the form $a + BY_t$, the minimum mean-square estimator under the moments in (2.7)–(2.9) uses*

$$B = C_{xz,t} S_t^{-1}, \qquad a = m_t^- - B\bar{z}_t. \tag{2.12}$$

Storing the resulting mean and covariance as a Gaussian gives (2.11).

Proof. Center $\widetilde{X} = X_t^- - m_t^-$ and $\widetilde{Y} = Y_t - \bar{z}_t$. The squared-error objective for $m_t^- + B\widetilde{Y}$ is

$$J(B) = \mathbb{E}\|\widetilde{X} - B\widetilde{Y}\|^2 = \operatorname{tr}(P_t^-) - 2\operatorname{tr}(BC_{xz,t}^\top) + \operatorname{tr}(BS_tB^\top). \tag{2.13}$$

Differentiating gives $\nabla_B J = -2C_{xz,t} + 2BS_t$. If S_t is nonsingular, the stationary point is $B = C_{xz,t}S_t^{-1}$. Substitution gives the residual covariance $P_t^- - C_{xz,t}S_t^{-1}C_{xz,t}^\top$, which equals (2.11). □

The approximation has two layers. First, the moments (2.7)–(2.9) may be approximated by quadrature. Second, the resulting posterior information is compressed to one Gaussian.

2.1.2 How This Matches The Jia–Xin–Cheng Gaussian Approximation Block

Jia, Xin, and Cheng first write the Gaussian approximation filter before they introduce sparse grids [1, Section 2.2]. In their additive-noise setting, the prediction moment equations have the same structure as

$$m_t^- = \mathbb{E}[f_\theta(X_{t-1})], \tag{2.14}$$
$$P_t^- = Q_\theta + \mathbb{E}[(f_\theta(X_{t-1}) - m_t^-)(f_\theta(X_{t-1}) - m_t^-)^\top], \tag{2.15}$$

where $X_{t-1} \sim \mathcal{N}(m_{t-1}, P_{t-1})$ under the carried Gaussian approximation. The update moment equations have the structure

$$\bar{z}_t = \mathbb{E}[h_\theta(X_t^-)], \tag{2.16}$$
$$S_t = R_\theta + \mathbb{E}[(h_\theta(X_t^-) - \bar{z}_t)(h_\theta(X_t^-) - \bar{z}_t)^\top], \tag{2.17}$$
$$C_{xz,t} = \mathbb{E}[(X_t^- - m_t^-)(h_\theta(X_t^-) - \bar{z}_t)^\top]. \tag{2.18}$$

Then the Gaussian projection update is

$$K_t = C_{xz,t} S_t^{-1}, \tag{2.19}$$
$$m_t = m_t^- + K_t(y_t - \bar{z}_t), \tag{2.20}$$
$$P_t = P_t^- - K_t S_t K_t^\top. \tag{2.21}$$

The sparse-grid part of the paper answers only how to approximate the expectations in (2.14)–(2.18). It does not change the fact that the filter stores (m_t, P_t). In BayesFilter notation, the replacement is

$$\mathbb{E}[F(\xi)] \quad \rightsquigarrow \quad A_{b,L}[F] \quad \rightsquigarrow \quad \sum_{r=1}^{M_{b,L}} w_r F(\xi^{(r)}). \tag{2.22}$$

For prediction,

$$F_{\text{pred}}(\xi) = f_\theta(m_{t-1} + C_{t-1}\xi),$$

and for observation,

$$F_{\text{obs}}(\xi) = h_\theta(m_t^- + C_t^-\xi).$$

This is the exact point where the source SGQF construction enters the filter. Everything before (2.22) is Gaussian projection; everything after it is fixed sparse-grid implementation of the needed Gaussian expectations. Before formalizing the quadrature machinery, it helps to see two small motivating cases that make the computational problem concrete.

3 Low-Dimensional Construction Of FixedSGQF

This chapter develops the sparse-grid construction in the order a reader usually needs it. Start from one nonlinear Gaussian moment in one coordinate. Lift that rule to a tensor-product cloud in two coordinates. Then ask how the same construction becomes too expensive in higher dimension and how the sparse-grid combination reduces that cost without changing the underlying standardized-coordinate logic.

3.1 One nonlinear Gaussian moment as the basic quadrature problem

Start with a scalar Gaussian input and a simple nonlinear observation map. Let

$$X \sim \mathcal{N}(m, P), \qquad h(X) = X^2. \tag{3.1}$$

Suppose we want the Gaussian-projection observation mean

$$\bar{z} = \mathbb{E}[h(X)] = \mathbb{E}[X^2]. \tag{3.2}$$

Write $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, 1)$ and $C^2 = P$. Then

$$\bar{z} = \int_{\mathbb{R}} (m + C\xi)^2 \phi(\xi)\, \mathrm{d}\xi. \tag{3.3}$$

This is the first key point: once the state has been standardized, the moment problem becomes a standard-normal weighted integral. The quadrature rule is therefore not an extra modeling choice layered on top of nowhere. It is a numerical approximation to the Gaussian expectation in (3.3).

In this scalar example, a three-point symmetric rule already reproduces the first few Gaussian moments exactly, which is why rules such as $\{-\sqrt{3}, 0, \sqrt{3}\}$ appear naturally later. The purpose of the next section is to formalize this one-dimensional Gaussian rule.

3.2 Tensor-product growth in two dimensions

Now move from one coordinate to two. Use a simple two-dimensional Gaussian state,

$$X \sim \mathcal{N}(m, P), \qquad X \in \mathbb{R}^2, \tag{3.4}$$

and imagine either a two-dimensional linear-Gaussian state-space step or a slightly nonlinear observation moment built from that same Gaussian input. The point of this example is geometric rather than inferential: it shows how the same standardized cloud is placed and why tensor products grow quickly.

Let $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, I_2)$. If each standardized coordinate uses the three-point rule $\{-\sqrt{3}, 0, \sqrt{3}\}$, then the direct tensor product uses all pairs

$$(\xi_1, \xi_2) \in \{-\sqrt{3}, 0, \sqrt{3}\} \times \{-\sqrt{3}, 0, \sqrt{3}\}, \tag{3.5}$$

so already in two dimensions the rule has $3^2 = 9$ points. In three dimensions the same construction has $3^3 = 27$ points; in ten dimensions it has 3^{10} points. This is the second key point: the tensor-product rule is simple and transparent, but its point count grows too quickly in higher dimension.

Sparse-grid construction tries to keep the same standardized-coordinate logic while avoiding the full tensor explosion. It does that by combining lower-level tensor rules with signed coefficients, then merging duplicate standardized nodes. The next two sections formalize that construction.

3.3 One-Dimensional Rules And Their Tensor Products

The scalar toy case above shows why a Gaussian expectation becomes a standard-normal weighted integral after standardization. This section now makes that one-dimensional rule explicit, and then shows how the same rule is lifted to multiple coordinates by tensor products.

How to read this section with the toy cases in mind. Toy case A should be kept in mind whenever a one-dimensional rule $I_\ell[\psi]$ appears: there the integrand is the scalar nonlinear moment in (3.3). Toy case B should be kept in mind whenever a tensor product appears: there the integrand lives on two standardized coordinates and the point set is the direct Cartesian product in (3.5). The formal notation below is therefore not new mathematics disconnected from the examples; it is the abstract version of those two simple cases.

3.3.1 The one-dimensional Gaussian quadrature rule

In Toy case A, we needed the scalar nonlinear Gaussian moment

$$\bar{z} = \int_{\mathbb{R}} (m + C\xi)^2 \phi(\xi)\, \mathrm{d}\xi. \tag{3.6}$$

The problem is simple to state: the integrand is nonlinear, so we want a small weighted sum that approximates this standard-normal integral without evaluating it analytically every time. A one-dimensional standard-normal quadrature rule at level ℓ is exactly such a weighted sum:

$$I_\ell[\psi] = \sum_{a=1}^{s_\ell} \omega_{\ell,a} \psi(\zeta_{\ell,a}) \approx \int_{\mathbb{R}} \psi(\xi)\phi(\xi)\, \mathrm{d}\xi. \tag{3.7}$$

Here:

- ψ is the function we want to integrate,
- $\zeta_{\ell,a}$ are the standardized evaluation points, or nodes,
- $\omega_{\ell,a}$ are the corresponding weights, and
- s_ℓ is the number of nodes used by level ℓ.

In Toy case A, the role of ψ is played by

$$\psi(\xi) = (m + C\xi)^2. \tag{3.8}$$

So equation (3.7) just says: approximate the scalar Gaussian moment in (3.6) by evaluating $(m+C\xi)^2$ at a small list of standardized points and taking a weighted sum.

Why do nodes and weights exist at all? They are not arbitrary decoration. They are chosen so that the weighted sum gets important Gaussian moments right. For example, a symmetric three-point rule can be chosen so that it integrates constants, ξ^2, and ξ^4 exactly under the standard-normal weight. That is why the later moment-matching equations matter: they explain where the concrete nodes and weights come from.

What is a standard-normal Gauss–Hermite rule? A standard-normal Gauss–Hermite quadrature rule is a deterministic weighted-sum approximation to a standard-normal integral,

$$\int_{\mathbb{R}} \psi(\xi)\phi(\xi)\,\mathrm{d}\xi,$$

chosen so that the rule is exact for as many low-degree polynomials as possible for its number of points. Concretely, one picks nodes $\zeta_{\ell,a}$ and weights $\omega_{\ell,a}$, evaluates the integrand at those nodes, and forms the weighted sum

$$\sum_{a=1}^{s_\ell} \omega_{\ell,a}\psi(\zeta_{\ell,a}).$$

The reason this family is important here is simple: once a Gaussian moment has been standardized to the coordinate ξ, standard-normal GHQ gives a canonical way to approximate that moment by a small deterministic set of points. In this note, the first few members are the familiar odd-point rules centered at zero: one point for level 1, three points for level 2, five points for level 3, and so on.

Which concrete family is being used in this note? At the teaching level, the easiest way to read the symbols is the following. In this note, the default BayesFilter family is the standard-normal GHQ ladder:

$I_1 = 1$-point standard-normal GHQ, $I_2 = 3$-point standard-normal GHQ, $I_3 = 5$-point standard-normal G

So the level label is a rule-family label, not a polynomial degree. Level 2 does not mean "degree 2 polynomial. It means "the second low-level member of the chosen one-dimensional GHQ family, which in this note is the familiar symmetric three-point rule. The later formal policy statement (3.60) records the same choice more precisely.

Where does the level-1 rule $I_1 : (0, 1)$ come from? It is the simplest one-dimensional standard-normal rule: one node at the mean $\xi = 0$ with total weight 1. In the notation of (3.7), this means $s_1 = 1$, $\zeta_{1,1} = 0$, and $\omega_{1,1} = 1$, so

$$I_1[\psi] = \psi(0).$$

This rule already integrates constants exactly, because

$$\int_{\mathbb{R}} 1 \cdot \phi(\xi)\, \mathrm{d}\xi = 1.$$

So $I_1 : (0, 1)$ is not mysterious notation. It is the natural level-1 base rule: evaluate at the center of the standardized Gaussian and assign it the full mass.

3.3.2 A fully worked one-dimensional toy rule

Take the symmetric three-point ansatz

$$\{(-a, \omega), (0, \omega_0), (a, \omega)\}. \tag{3.9}$$

To match the first relevant Gaussian moments, require

$$\omega_0 + 2\omega = 1, \tag{3.10}$$
$$2\omega a^2 = 1, \tag{3.11}$$
$$2\omega a^4 = 3. \tag{3.12}$$

Dividing (3.12) by (3.11) gives $a^2 = 3$. Then $\omega = 1/6$ and $\omega_0 = 2/3$. So the rule becomes

$$\left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{3.13}$$

This solved three-point rule is exactly the later level-2 rule I_2 used in the two-dimensional and three-dimensional examples. In other words, the notation I_2 is not a new assumption introduced later: it is simply shorthand for the symmetric three-point standard-normal rule obtained from (3.10)–(3.12). Together with the level-1 rule $I_1 : (0, 1)$, it gives the smallest pair of one-dimensional rules needed for the worked tensor examples below.

Returning to Toy case A, the scalar moment is then approximated by

$$\bar{z} \approx \frac{1}{6}(m - C\sqrt{3})^2 + \frac{2}{3}m^2 + \frac{1}{6}(m + C\sqrt{3})^2. \tag{3.14}$$

This is the simplest concrete instance of Gaussian quadrature used anywhere in this note.

3.3.3 Lifting the rule to two dimensions

Now move to Toy case B. Instead of one standardized coordinate ξ, we now have two coordinates (ξ_1, ξ_2). Suppose we want to approximate a Gaussian-weighted integral of a function $F(\xi_1, \xi_2)$. If we reuse the same three-point rule in each coordinate, the direct tensor-product rule places all pairs of one-dimensional nodes. That gives the nine-point grid already listed in (3.24).

At this stage there is still no sparse-grid machinery. We are only taking the one-dimensional rule and using it in both coordinates. The reader should think of the tensor-product construction as a rectangular grid in standardized space.

What does the symbol F mean? When the note later writes expressions such as $I_{\mathbf{i}}[F]$ or $A_{b,L}[F]$, the symbol F is just the multivariate integrand being averaged under the Gaussian weight. In one dimension the same role is often played by ψ. In several coordinates we simply switch to $F(\xi)$ to remind the reader that the function may depend on many variables. A concrete example to keep in mind is

$$F(\xi_1, \xi_2, \xi_3) = e^{\xi_1 \xi_2 \xi_3},$$

which will be reused later to illustrate what a low sparse-grid level can and cannot see.

3.3.4 What is the multi-index $\mathbf{i} = (i_1, \ldots, i_b)$?

The multi-index arrives because different coordinates may use different one-dimensional rule levels. For example:

- $(1, 1)$ means "use level 1 in coordinate 1 and level 1 in coordinate 2,"
- $(1, 2)$ means "use level 1 in coordinate 1 and level 2 in coordinate 2,"
- $(2, 1)$ means "use level 2 in coordinate 1 and level 1 in coordinate 2,"
- $(2, 2)$ means "use level 2 in both coordinates."

So the multi-index does not come from nowhere. It is simply a bookkeeping device that records which one-dimensional rule level is used in each coordinate.

Formally, for a multi-index $\mathbf{i} = (i_1, \ldots, i_b)$, the corresponding tensor rule is

$$I_{\mathbf{i}}[F] = (I_{i_1} \otimes \cdots \otimes I_{i_b})[F]. \tag{3.15}$$

This means:

> apply the one-dimensional rule of level i_1 in the first coordinate, the one-dimensional rule of level i_2 in the second coordinate, and so on, then take the full Cartesian product of the resulting nodes and weights.

So the tensor rule is just the multi-coordinate version of the scalar weighted sum from (3.7). First choose a one-dimensional rule in each coordinate. Then list every possible combination of those coordinatewise nodes. Finally, attach to each combined point the product of the corresponding one-dimensional weights. The tensor notation therefore does not introduce a new kind of quadrature object; it only packages the coordinatewise Cartesian-product construction compactly.

3.3.5 A fully worked two-dimensional tensor example

Take $b = 2$ and use:

$$I_1 : (0, 1), \qquad I_2 : \left\{ \left(-\sqrt{3}, \frac{1}{6} \right), \left(0, \frac{2}{3} \right), \left(\sqrt{3}, \frac{1}{6} \right) \right\}. \tag{3.16}$$

Now work out three explicit tensor rules.

Rule $(1, 1)$. Both coordinates use the one-point rule, so there is only one tensor point:

$$(0, 0) \quad \text{with weight} \quad 1 \cdot 1 = 1. \tag{3.17}$$

Rule $(1,2)$**.** The first coordinate uses level 1 and the second uses level 2. So the tensor points are:

$$(0,-\sqrt{3}), \qquad (0,0), \qquad (0,\sqrt{3}), \tag{3.18}$$

with weights

$$\frac{1}{6}, \qquad \frac{2}{3}, \qquad \frac{1}{6}. \tag{3.19}$$

Rule $(2,1)$**.** Now the first coordinate uses level 2 and the second uses level 1. So the tensor points are:

$$(-\sqrt{3},0), \qquad (0,0), \qquad (\sqrt{3},0), \tag{3.20}$$

with weights

$$\frac{1}{6}, \qquad \frac{2}{3}, \qquad \frac{1}{6}. \tag{3.21}$$

This is the concrete meaning of the tensor-rule notation. The symbols $\xi^{\mathbf{i},\alpha}$ and $w^{\mathbf{i},\alpha}$ introduced below are just a compact way to name these explicit tensor points and their product weights.

Its nodes and weights are

$$\xi^{\mathbf{i},\alpha} = (\zeta_{i_1,\alpha_1}, \ldots, \zeta_{i_b,\alpha_b}), \tag{3.22}$$

$$w^{\mathbf{i},\alpha} = \prod_{j=1}^{b} \omega_{i_j,\alpha_j}, \qquad \alpha_j \in \{1,\ldots,s_{i_j}\}. \tag{3.23}$$

Equation (3.22) says how to build one tensor point: choose one standardized node from each coordinate rule and place those chosen values next to each other. Equation (3.23) then says how to weight that point: multiply the weights attached to the chosen one-dimensional nodes.

In the concrete $(2,2)$ case, each coordinate uses the level-2 rule $I_2 = \{(-\sqrt{3},1/6),(0,2/3),(\sqrt{3},1/6)\}$. So $\alpha_1,\alpha_2 \in \{1,2,3\}$, and every pair (α_1,α_2) chooses one node from coordinate 1 and one node from coordinate 2. For example,

$$(\alpha_1,\alpha_2) = (1,1) \mapsto (-\sqrt{3},-\sqrt{3}) \quad \text{with weight} \quad \frac{1}{6}\cdot\frac{1}{6} = \frac{1}{36},$$

$$(\alpha_1,\alpha_2) = (2,2) \mapsto (0,0) \quad \text{with weight} \quad \frac{2}{3}\cdot\frac{2}{3} = \frac{4}{9},$$

and

$$(\alpha_1,\alpha_2) = (3,1) \mapsto (\sqrt{3},-\sqrt{3}) \quad \text{with weight} \quad \frac{1}{6}\cdot\frac{1}{6} = \frac{1}{36}.$$

So equation (3.24) is not a different construction from (3.22)–(3.23). It is simply those two abstract formulas written out explicitly for the case $b = 2$ with level 2 in both coordinates.

In Toy case B with two coordinates and the same three-point rule in each coordinate, this abstract tensor product is exactly the nine-point Cartesian product already listed in (3.5). The notation $\xi^{\mathbf{i},\alpha}$ is just the formal way to say "take one selected standardized point from coordinate 1 and one selected standardized point from coordinate 2, then pair them."

It helps to see the 2D case written out explicitly. If both coordinates use the three-point rule $\{-\sqrt{3},0,\sqrt{3}\}$ with weights $\{1/6,2/3,1/6\}$, then the tensor-product rule places the nine points

$$(-\sqrt{3},-\sqrt{3}),(-\sqrt{3},0),(-\sqrt{3},\sqrt{3}),(0,-\sqrt{3}),(0,0),(0,\sqrt{3}),(\sqrt{3},-\sqrt{3}),(\sqrt{3},0),(\sqrt{3},\sqrt{3}), \tag{3.24}$$

with weights given by products of the one-dimensional weights. For example, $(0, 0)$ has weight $(2/3)(2/3) = 4/9$, while $(\sqrt{3}, -\sqrt{3})$ has weight $(1/6)(1/6) = 1/36$. Nothing mysterious has happened yet: tensor products just place all combinations of one-dimensional points.

A full tensor-product rule is simple but expensive: if every coordinate uses s nodes, then $M = s^b$. The two-dimensional toy case above already shows this growth geometrically. With three points per coordinate, two dimensions use nine points, three dimensions use twenty-seven points, and ten dimensions use 3^{10} points. Jia, Xin, and Cheng introduce the sparse-grid combination to reduce this point count for fixed accuracy level [1, Section 3].

3.4 Three-dimensional preview: why sparse grids help

The panel asked for one concrete three-dimensional case showing what sparse-grid combination is trying to buy. Here is the cleanest version.

Use the same one-dimensional rules as before:

$$I_1 : (0, 1), \qquad I_2 : \left\{ \left(-\sqrt{3}, \frac{1}{6} \right), \left(0, \frac{2}{3} \right), \left(\sqrt{3}, \frac{1}{6} \right) \right\}. \tag{3.25}$$

Here "level 2" means: in each one-dimensional coordinate, use the level-2 rule I_2, namely the three-point standard-normal rule with nodes $-\sqrt{3}, 0, \sqrt{3}$ and weights $1/6, 2/3, 1/6$. So the product rule $(2, 2, 2)$ is only a bookkeeping label saying

> use I_2 in coordinate 1, use I_2 in coordinate 2, and use I_2 in coordinate 3.

Because each coordinate contributes three one-dimensional nodes, the full tensor product has $3 \times 3 \times 3 = 27$ raw node combinations before any sparse-grid selection or duplicate merging.

If all three coordinates use level 2, then the full tensor-product rule $(2, 2, 2)$ places

$$3 \times 3 \times 3 = 27 \tag{3.26}$$

standardized points. That is the direct three-dimensional analogue of the nine-point 2D grid.

Now compare that with the sparse-grid band for $b = 3$ and $L = 2$. This is a different use of the word "level." The tensor-product label $(2, 2, 2)$ means "use the one-dimensional level-2 rule in every coordinate." By contrast, the sparse-grid level $L = 2$ means: combine all tensor rules whose total index lies in the admissible band for $(b, L) = (3, 2)$. So the sparse-grid construction does not keep the full 27-point tensor rule $(2, 2, 2)$. Instead, it mixes several lower-total-index tensor rules with signed coefficients.

The allowed multi-indices satisfy

$$2 \le |\mathbf{i}| \le 4. \tag{3.27}$$

Since every coordinate level is at least 1, the only admissible three-dimensional multi-indices are

$$(1, 1, 1), \qquad (1, 1, 2), \qquad (1, 2, 1), \qquad (2, 1, 1). \tag{3.28}$$

So the sparse-grid rule does not use the full level-2 tensor product $(2, 2, 2)$. Instead it combines one one-point tensor rule and three three-point tensor rules. The corresponding Smolyak coefficients are

$$c_{(1,1,1)} = -2, \qquad c_{(1,1,2)} = 1, \qquad c_{(1,2,1)} = 1, \qquad c_{(2,1,1)} = 1. \tag{3.29}$$

So even in this concrete case the sparse-grid rule is already a signed combination of lower-level tensor rules rather than a simple subset of the 27-point tensor product.

To make the reduction feel less abstract, fix a simple explicit test function, for example

$$F(\xi_1, \xi_2, \xi_3) = \xi_1^2 + \xi_2^2 + \xi_3^2. \tag{3.30}$$

This is a good teaching example because it depends only on single-coordinate quadratic terms. It therefore matches the kind of low-order interaction structure that a low sparse-grid level is designed to preserve.

Under the full 27-point tensor rule $(2, 2, 2)$, the weighted average of F is exactly

$$\mathbb{E}[F(\xi)] = \mathbb{E}[\xi_1^2] + \mathbb{E}[\xi_2^2] + \mathbb{E}[\xi_3^2] = 1 + 1 + 1 = 3. \tag{3.31}$$

Now check the sparse-grid construction on the same function. On the merged seven-point cloud described below, the center point contributes $F(0, 0, 0) = 0$, while each axis point such as $(\sqrt{3}, 0, 0)$ or $(0, -\sqrt{3}, 0)$ gives

$$F = (\sqrt{3})^2 = 3. \tag{3.32}$$

If the seven-point merged cloud has center weight 0 and six axis weights $1/6$, then the sparse-grid sum is

$$\sum_r w_r F(\xi^{(r)}) = 6 \cdot \frac{1}{6} \cdot 3 = 3, \tag{3.33}$$

which exactly matches (3.31). So in this concrete case, the sparse-grid rule keeps far fewer points than the 27-point tensor product but still gets the selected low-order quadratic structure exactly right.

The raw point counts before duplicate merging are therefore:

$$(1, 1, 1) \rightsquigarrow 1 \text{ point}, \tag{3.34}$$
$$(1, 1, 2) \rightsquigarrow 3 \text{ points}, \tag{3.35}$$
$$(1, 2, 1) \rightsquigarrow 3 \text{ points}, \tag{3.36}$$
$$(2, 1, 1) \rightsquigarrow 3 \text{ points}, \tag{3.37}$$

for a total of

$$1 + 3 + 3 + 3 = 10 \tag{3.38}$$

raw contributions before duplicate merging. The point $(0, 0, 0)$ appears in all four tensor components, so after duplicate merging the cloud has seven distinct nodes:

$$(0, 0, 0), \qquad (\pm\sqrt{3}, 0, 0), \qquad (0, \pm\sqrt{3}, 0), \qquad (0, 0, \pm\sqrt{3}), \tag{3.39}$$

which is only

$$1 + 2 + 2 + 2 = 7 \tag{3.40}$$

distinct points before any zero-weight pruning rule is applied.

Because the merged center weight is exactly zero in this worked level-2 case, the stored fixed cloud defined later by the zero-weight pruning convention keeps only the six axis points. So there are three distinct 3D objects to keep separate:

1. the 27-point full tensor-product comparator $(2, 2, 2)$,
2. the 10 raw sparse-grid contributions from $(1, 1, 1), (1, 1, 2), (1, 2, 1), (2, 1, 1)$, and
3. the final stored 6-point cloud after duplicate merging and zero-weight pruning.

So in this concrete 3D case the sparse-grid construction replaces the direct 27-point tensor-product picture by a signed sparse-grid combination with 10 raw contributions, 7 merged distinct nodes, and 6 stored nodes after pruning. Why does this work? Because the low sparse-grid level keeps only low-order tensor interactions. Here, level 2 in three dimensions keeps the center point and one-axis excursions, but it does not yet spend points on all pairwise or all three-way corner combinations. This is exactly the intended economy of the sparse-grid rule: pay for lower-order interaction structure before paying for the full tensor product.

The next section now formalizes this economy in general notation.

3.4.1 Sparse-Grid Rule Reconstructed In Source Order

The sparse-grid construction answers a specific question raised by the toy cases. The scalar example shows why nonlinear Gaussian moments become standard-normal weighted integrals after standardization. The two-dimensional example shows why naive tensor products grow quickly even when the placement logic is simple. The purpose of sparse-grid combination is to keep the standardized-coordinate moment logic while reducing the point count relative to the full tensor product.

How to read this section with the toy cases in mind. Keep returning mentally to the worked low-dimensional examples from the previous section. In two dimensions, we explicitly listed the tensor rules $(1,1)$, $(1,2)$, and $(2,1)$, and we saw the resulting points. In three dimensions, we now also have the concrete preview in which the full 27-point tensor-product picture is reduced to a 7-point merged cloud. This section asks how to turn those concrete constructions into one scalable general rule.

3.4.2 Start with the scalar and 2D cases before the general notation

In one dimension, there is no sparse-grid combination to explain. One simply chooses a one-dimensional Gaussian rule and uses it. The scalar toy problem is therefore the basic building block.

In two dimensions, however, one already has several tensor rules available. For example, with the level-1 and level-2 rules from (3.16), the tensor rules $(1,1)$, $(1,2)$, and $(2,1)$ are all explicit point sets. A sparse-grid rule will not keep all possible tensor levels equally. Instead, it chooses a structured subset of those tensor rules and combines them with signed coefficients.

The general notation below is just the scalable version of that statement.

Suppose the filter needs a Gaussian expectation

$$\mathcal{I}_b[F] = \int_{\mathbb{R}^b} F(\xi)\phi_b(\xi)\,\mathrm{d}\xi, \qquad \phi_b(\xi) = (2\pi)^{-b/2}\exp(-\|\xi\|^2/2). \tag{3.41}$$

In the scalar toy case, this is just the one-dimensional standard-normal integral in (3.3). In the 2D toy case, it is a Gaussian-weighted integral over (ξ_1, ξ_2), approximated first by tensor rules such as $(1,1)$, $(1,2)$, and $(2,1)$.

A full tensor rule approximates $\mathcal{I}_b[F]$ by evaluating F on every Cartesian product of one-dimensional nodes. If each coordinate uses s points, the point count is s^b. Jia, Xin, and Cheng replace that full tensor rule by a signed linear combination of lower-level tensor rules [1, Section 3.1]. The signs are not an implementation accident. They are inclusion-exclusion coefficients: low-level tensor rules overlap, and overlapping contributions must be subtracted before higher-level contributions are added.

3.4.3 The Jia–Xin–Cheng Index Sets

Jia, Xin, and Cheng group tensor-product levels by an auxiliary integer q [1, Eq. 27]. In BayesFilter notation, for dimension b,

$$N_q^b = \left\{ \mathbf{i} = (i_1, \dots, i_b) \in \mathbb{N}^b : \sum_{j=1}^{b} i_j = b + q \right\}, \qquad q \geq 0, \tag{3.42}$$

and $N_q^b = \emptyset$ for $q < 0$. A member $\mathbf{i} \in N_q^b$ says: use the one-dimensional rule I_{i_j} in coordinate j, then take their tensor product. The larger q is, the more coordinates can have levels above one.

This is the abstract version of the explicit 2D examples from the previous section. When $b = 2$:

- $(1, 1)$ lies in the slice where the total index is 2,
- $(1, 2)$ and $(2, 1)$ lie in the slice where the total index is 3,
- $(2, 2)$ would lie in the slice where the total index is 4.

So the set N_q^b is simply a way to group multi-indices by their total level. It is not a new probabilistic object; it is a bookkeeping device for organizing which tensor rules are being combined.

In the 3D preview from the previous section, the same symbols mean the following. Take:

$$b = 3, \qquad q \in \{0, 1\}, \qquad \mathbf{i} = (i_1, i_2, i_3). \tag{3.43}$$

Then:

- $b = 3$ means there are three standardized coordinates,
- q says how far the total index is above the baseline value b,
- N_q^3 is the set of all three-coordinate multi-indices with total sum $3 + q$,
- $\mathbf{i} = (i_1, i_2, i_3)$ records which one-dimensional rule level is used in each of the three coordinates.

So, for example,

$$N_0^3 = \{(1, 1, 1)\}, \tag{3.44}$$

$$N_1^3 = \{(1, 1, 2), (1, 2, 1), (2, 1, 1)\}. \tag{3.45}$$

These are exactly the 3D tensor rules used in the preview: one central one-point tensor rule and three one-axis three-point tensor rules. This is the most concrete way to read the notation N_q^b in the present note.

For a declared sparse-grid level L, the source sums over

$$q = L - b,\ L - b + 1, \dots, L - 1.$$

Since N_q^b is empty when $q < 0$, this formula also covers the case $L < b$. If $\mathbf{i} \in N_q^b$, then

$$|\mathbf{i}| = \sum_{j=1}^{b} i_j = b + q. \tag{3.46}$$

Thus the same index set can be written without q as

$$\mathcal{A}_{b,L} = \{\mathbf{i} \in \mathbb{N}^b : L \leq |\mathbf{i}| \leq L + b - 1\}, \qquad |\mathbf{i}| = \sum_{j=1}^{b} i_j. \tag{3.47}$$

Important warning about the word "level. This is the place where many readers first get lost. The sparse-grid level $L = 2$ is *not* the same object as the tensor rule $(2, 2)$ or the three-dimensional tensor rule $(2, 2, 2)$. The tensor label $(2, 2)$ says "use the level-2 one-dimensional rule in both coordinates. By contrast, the sparse-grid level $L = 2$ says "keep tensor rules whose total index lies in the active band for level 2 and combine them by the Smolyak rule. That is why the active two-dimensional sparse-grid set contains $(1, 1)$, $(1, 2)$, and $(2, 1)$, but not $(2, 2)$.

This is exactly the band used by the implementable fixed-cloud construction in this note.

To keep the 2D toy case in view, take $b = 2$ and $L = 2$. Then the admissible band becomes:

- $(1, 1)$, because $|(1, 1)| = 2$,
- $(1, 2)$, because $|(1, 2)| = 3$,
- $(2, 1)$, because $|(2, 1)| = 3$,
- but not $(2, 2)$, because $|(2, 2)| = 4$ lies outside the band for this choice of L.

So in the 2D case the active band already recovers the three tensor rules that will later build the five-point merged toy cloud. This is the concrete meaning of the abstract band definition.

In the 3D case with $b = 3$ and $L = 2$, the same band definition gives

$$\mathcal{A}_{3,2} = \{(1, 1, 1), (1, 1, 2), (1, 2, 1), (2, 1, 1)\}, \tag{3.48}$$

because those are exactly the multi-indices whose total level lies between 2 and $2 + 3 - 1 = 4$ while still respecting that every coordinate level is at least 1. So the abstract band notation now directly recovers the explicit 3D preview from the previous section.

3.4.4 The Smolyak Coefficients

In the source formula, the coefficient attached to the q-slice is

$$(-1)^{L-1-q}\binom{b-1}{L-1-q}.$$

Using $q = |\mathbf{i}| - b$ from (3.46), this becomes

$$c_{\mathbf{i}} = (-1)^{L+b-1-|\mathbf{i}|}\binom{b-1}{L+b-1-|\mathbf{i}|}. \tag{3.49}$$

Therefore the sparse-grid quadrature approximation is

$$A_{b,L}[F] = \sum_{\mathbf{i}\in\mathcal{A}_{b,L}} c_{\mathbf{i}} I_{\mathbf{i}}[F]. \tag{3.50}$$

Equation (3.50) is the BayesFilter form of Jia et al. [1, Eq. 26–29]. The coefficient (3.49) can be negative. That fact later explains why the filter must check covariance matrices instead of assuming every weighted second-moment sum is automatically positive semidefinite.

In the concrete 2D case with $L = 2$, the active tensor rules are $(1, 1)$, $(1, 2)$, and $(2, 1)$. Their coefficients turn out to be

$$c_{(1,1)} = -1, \qquad c_{(1,2)} = 1, \qquad c_{(2,1)} = 1. \tag{3.51}$$

So the sparse rule in this case is not "keep some of the nine tensor-product points and drop the others." It is

$$A_{2,2}[F] = -I_{(1,1)}[F] + I_{(1,2)}[F] + I_{(2,1)}[F]. \tag{3.52}$$

This is the place where the construction first looks strange to many readers, and it is worth stating the point slowly: the sparse-grid rule is a signed combination of tensor rules, not a simple pruning of one big tensor grid. Because the component tensor rules overlap, some repeated point contributions must later be added together and some net contributions become smaller than the raw tensor pieces suggest.

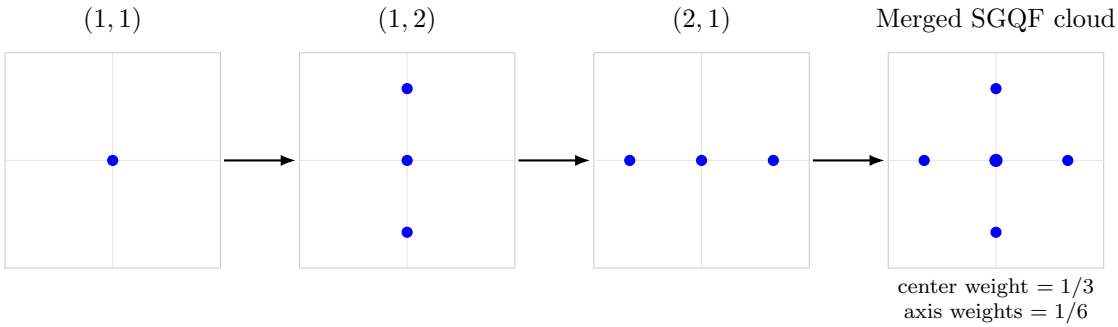

Figure 1: How the two-dimensional sparse-grid cloud is built. The active tensor rules $(1,1)$, $(1,2)$, and $(2,1)$ are expanded separately, then combined with Smolyak coefficients and merged. This makes visible the fact that the final SGQF cloud is assembled from several smaller tensor rules rather than chosen directly as one single point set.

The same story can now be read in the 3D case. Using (3.48), the active rules are $(1,1,1)$, $(1,1,2)$, $(1,2,1)$, and $(2,1,1)$. Their coefficients are

$$c_{(1,1,1)} = -2, \qquad c_{(1,1,2)} = 1, \qquad c_{(1,2,1)} = 1, \qquad c_{(2,1,1)} = 1. \tag{3.53}$$

So the 3D sparse rule is

$$A_{3,2}[F] = -2I_{(1,1,1)}[F] + I_{(1,1,2)}[F] + I_{(1,2,1)}[F] + I_{(2,1,1)}[F]. \tag{3.54}$$

This is the formal equation behind the earlier statement that the 27-point tensor product is being replaced by a signed combination of one one-point tensor rule and three one-axis three-point tensor rules.

3.4.5 Univariate Moment Matching

The one-dimensional rules I_ℓ must be chosen before (3.50) becomes a concrete point set. Jia, Xin, and Cheng construct univariate rules by matching moments of a standard normal [1, Section 3.2]. If p_a and ω_a are the one-dimensional nodes and weights, the matching equations are

$$M_j = \int_{-\infty}^{\infty} x^j \phi(x)\,\mathrm{d}x = \sum_{a=1}^{N_u} \omega_a p_a^j, \qquad j = 0, 1, \ldots, J. \tag{3.55}$$

The moments on the left are explicit, with the usual convention $M_0 = 1$ and the empty double-factorial product taken to equal one:

$$M_j = \begin{cases} 0, & j \text{ odd}, \\ (j-1)(j-3)\cdots 3 \cdot 1, & j \text{ even}. \end{cases} \tag{3.56}$$

For a symmetric three-point rule

$$\{(-a,\omega),(0,\omega_0),(a,\omega)\},$$

the first three nontrivial equations are

$$\omega_0 + 2\omega = 1, \tag{3.57}$$

$$2\omega a^2 = 1, \tag{3.58}$$

$$2\omega a^4 = 3. \tag{3.59}$$

Dividing (3.59) by (3.58) gives $a^2 = 3$. Then $\omega = 1/6$ and $\omega_0 = 2/3$. This derivation is why the toy cloud below uses nodes $\{-\sqrt{3}, 0, \sqrt{3}\}$ with weights $\{1/6, 2/3, 1/6\}$. In the source SGQ family, however, the level-2 three-point rule is tunable through the location parameter shown in Jia 2012 Eq. (34)–(35); this note fixes the Gauss–Hermite specialization $\hat{p}_1 = \sqrt{3}$ so that the worked toy rules, the 3D preview, and the level-2 UKF bridge all use one explicit three-point reference rule. The same moment-matching logic extends to the higher level symmetric rules discussed by Jia et al. [1, Section 3.2].

Which univariate family is fixed in this note? The source theorem does not force one unique formula for every level. What it requires is a sequence of one-dimensional rules $\mathcal{I} = \{I_\ell : \ell \in \mathbb{N}\}$ such that the level-ℓ rule is exact for univariate polynomials of degree at least $2\ell - 1$ under the standard-normal weight. Many families can satisfy that property, and Jia, Xin, and Cheng explicitly allow other point-selection methods as long as the required exactness is met.

This note therefore makes one additional choice that the source itself does not force: it fixes the univariate family to the odd-point standard-normal Gauss–Hermite rules,

$$I_1 = \text{1-point standard-normal GHQ}, \qquad I_2 = \text{3-point standard-normal GHQ}, \qquad I_3 = \text{5-point standard-normal G} \tag{3.60}$$

Here "standard-normal GHQ" means the Gaussian quadrature rule for the weight $\phi(\xi)$; equivalently, it is the classical Gauss–Hermite rule rescaled into standard-normal coordinates. Its n-point member integrates standard-normal weighted polynomials through degree $2n-1$ exactly, so choosing $n = 2\ell-1$ gives a level-ℓ rule that is actually exact through degree $4\ell - 3$, which is stronger than the source theorem's minimum requirement $2\ell - 1$.

This is not an accidental choice. It is a deliberate specialization made for four reasons. First, it satisfies the Jia–Xin–Cheng admissibility condition with margin. Second, it gives the note one concrete deterministic family rather than a tunable family left implicit, which is important for the fixed-cloud and same-scalar contracts. Third, its first low levels are symmetric odd-point rules centered at zero, which makes the worked 1D, 2D, and 3D examples easy to teach and audit. Fourth, the familiar 1-point / 3-point / 5-point ladder is a canonical Gaussian-filtering reference family, so using it makes the sparse-grid construction easier to compare against dense GHQ and against the source's UKF bridge.

Moment matching is still the right concrete way to see how these low levels are constructed. The derivations above for I_1 and I_2 are not separate from this GHQ-family choice; they are the first two explicit members of the family in standard-normal coordinates. One could derive I_3 in the same style by matching more moments, but the present note does not need the full five-point formula because its worked sparse-grid examples use only levels 1 and 2.

This is also the exact place where Toy case A connects back to the formal rule. The scalar nonlinear moment in (3.3) needs a one-dimensional Gaussian rule. The three-point rule just derived

is the first concrete example of such a rule. Toy case B then reuses the same one-dimensional rule coordinatewise before tensor products or sparse-grid combinations are even considered. The 3D preview does the same thing once more: each coordinate still uses the same one-dimensional rule, but the combination logic changes because there are now more possible tensor directions.

3.4.6 From Formula To Point Cloud

Equation (3.50) is still not an implementation object. The implementation does not evaluate a symbolic formula directly; it must turn that formula into an explicit merged list of standardized nodes and weights. The 2D case is the right place to understand this slowly.

Step 1: list the contributing tensor rules. For $b = 2$ and $L = 2$, the active rules are $(1, 1)$, $(1, 2)$, and $(2, 1)$, with coefficients from (3.51).

Step 2: expand each tensor rule into explicit points.

- Rule $(1, 1)$ contributes only $(0, 0)$, with tensor weight 1, then sparse-grid coefficient -1.
- Rule $(1, 2)$ contributes $(0, -\sqrt{3})$, $(0, 0)$, and $(0, \sqrt{3})$, with tensor weights 1/6, 2/3, and 1/6, then sparse-grid coefficient $+1$.
- Rule $(2, 1)$ contributes $(-\sqrt{3}, 0)$, $(0, 0)$, and $(\sqrt{3}, 0)$, with tensor weights 1/6, 2/3, and 1/6, then sparse-grid coefficient $+1$.

Step 3: merge repeated standardized nodes. The node $(0, 0)$ appears in all three tensor components. Its accumulated weight is therefore

$$-1 + \frac{2}{3} + \frac{2}{3} = \frac{1}{3}. \tag{3.61}$$

The four axis nodes each appear only once, so they keep weight 1/6. The final merged cloud is exactly the five-point cloud presented later in the toy example section.

The same mechanism works in the 3D preview, only with one more coordinate. The active tensor rules are $(1, 1, 1)$, $(1, 1, 2)$, $(1, 2, 1)$, and $(2, 1, 1)$. They are not the full level-2 tensor rule $(2, 2, 2)$; they are the lower-total-index sparse-grid components whose signed combination replaces that 27-point comparator at level $L = 2$. Their explicit tensor points are:

- $(1, 1, 1)$: only $(0, 0, 0)$, with tensor weight 1 and sparse-grid coefficient -2,
- $(1, 1, 2)$: $(0, 0, -\sqrt{3})$, $(0, 0, 0)$, $(0, 0, \sqrt{3})$, with tensor weights 1/6, 2/3, 1/6,
- $(1, 2, 1)$: $(0, -\sqrt{3}, 0)$, $(0, 0, 0)$, $(0, \sqrt{3}, 0)$, with tensor weights 1/6, 2/3, 1/6,
- $(2, 1, 1)$: $(-\sqrt{3}, 0, 0)$, $(0, 0, 0)$, $(\sqrt{3}, 0, 0)$, with tensor weights 1/6, 2/3, 1/6.

So the origin again appears in every tensor component. Its accumulated 3D weight is

$$-2 + \frac{2}{3} + \frac{2}{3} + \frac{2}{3} = 0. \tag{3.62}$$

Thus the center point cancels completely at this level. The six axis points survive, each with weight 1/6, so the final merged 3D cloud is

$$(\pm\sqrt{3}, 0, 0), \qquad (0, \pm\sqrt{3}, 0), \qquad (0, 0, \pm\sqrt{3}). \tag{3.63}$$

This is the explicit 3D analogue of the 2D five-point merged cloud: after the signed combination and duplicate merge are done, the cloud has seven distinct nodes before zero-weight pruning and six stored nodes after the canceled center is removed.

Now return to the general notation. Each tensor rule $I_{\mathbf{i}}$ has points and weights

$$\xi^{\mathbf{i},\alpha} = (\zeta_{i_1,\alpha_1}, \ldots, \zeta_{i_b,\alpha_b}), \tag{3.64}$$

$$w^{\mathbf{i},\alpha} = \prod_{j=1}^{b} \omega_{i_j,\alpha_j}, \qquad \alpha_j \in \{1, \ldots, s_{i_j}\}. \tag{3.65}$$

The sparse-grid contribution of this tensor point is

$$c_{\mathbf{i}} w^{\mathbf{i},\alpha}. \tag{3.66}$$

The same standardized point can appear in several tensor components. Jia, Xin, and Cheng's Algorithm 1 says to add a new point if it has not appeared before, and otherwise update the old weight [1, Algorithm 1]. The mathematical object is therefore the accumulated dictionary

$$D[\xi] = \sum_{\substack{\mathbf{i} \in \mathcal{A}_{b,L} \\ \alpha:\, \xi^{\mathbf{i},\alpha} = \xi}} c_{\mathbf{i}} w^{\mathbf{i},\alpha}. \tag{3.67}$$

After merging duplicate standardized nodes, enumerate the nonzero entries as

$$\mathcal{C}_{b,L} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M_{b,L}}. \tag{3.68}$$

This report adopts the following deterministic cloud-construction convention.

1. Enumerate multi-indices $\mathbf{i} \in \mathcal{A}_{b,L}$ in lexicographic order.
2. Within each tensor rule $I_{\mathbf{i}}$, enumerate the local index tuples α lexicographically.
3. For each candidate standardized node $\xi^{\mathbf{i},\alpha}$, search the current dictionary for an existing node ξ satisfying
$$\|\xi^{\mathbf{i},\alpha} - \xi\|_\infty \le \tau_{\text{merge}} := 10^{-12} \max\{1, \|\xi\|_\infty\}. \tag{3.69}$$
If such a node exists, accumulate the signed weight $c_{\mathbf{i}} w^{\mathbf{i},\alpha}$ into its stored weight; otherwise add a new dictionary entry.
4. Remove entries whose accumulated absolute weight is below the declared numerical-zero threshold.
5. Sort the surviving merged nodes lexicographically and store the final cloud in that order.

For the fixed BayesFilter scalar, $\mathcal{C}_{b,L}$, the univariate rules, the merge tolerance, the zero-weight rule, the sorting convention, and the accumulated signed weights are part of the target. If two implementations use the same level L but merge nearly-equal floating-point nodes differently, they have not necessarily evaluated the same scalar.

The most important conceptual point is now visible from the worked 2D case. The cloud used later by the filter is not a symbolic Smolyak expression and not a raw tensor rule. It is the merged, sorted list of standardized nodes and accumulated signed weights that remains after all tensor components have been expanded and duplicates have been combined. That is why the later filter stores $\mathcal{C}_{b,L}$ rather than a list of symbolic index sets.

Implementation-facing pseudocode summary. The runtime object produced by this section is a stored cloud builder routine of the form `build_fixed_sparse_grid_cloud`$((b, L, \{I_\ell\}, \tau_{\text{merge}}, \tau_{\text{zero}}))$ that returns the sorted merged cloud $\mathcal{C}_{b,L}$, its weight-total diagnostic, and any signed-weight diagnostics used later in validation. The mathematics above is the authoritative definition of that routine.

3.4.7 Exactness, Point Count, And The UKF Relation

Theorem 3.1 of Jia et al. [1] is a quadrature exactness statement. If each univariate I_ℓ is exact for univariate polynomials through degree $2\ell - 1$, then the sparse-grid rule is exact for multivariate polynomials through the corresponding total degree stated in that theorem. The theorem supports this inference:

$$F \text{ polynomial in the exactness class} \quad \Longrightarrow \quad A_{b,L}[F] = \mathcal{I}_b[F]. \tag{3.70}$$

It does not support the stronger inference

nonlinear posterior is exactly represented by one Gaussian.

FixedSGQF uses (3.70) to justify deterministic moment approximation, not to claim exact Bayesian filtering.

The point-count proposition in Jia et al. [1, Proposition 3.1] also has a precise scope. For fixed level L, the number of sparse-grid points grows polynomially in dimension, with highest order $L - 1$ under the source conditions. The core counting reason is visible from (3.42): in the largest relevant slice, at most $L - 1$ coordinates can have levels above one. The number of ways to choose those coordinates is of order $\binom{b}{L-1}$, which is polynomial in b when L is fixed. This explains the storage advantage over full tensor products. It does not choose the correct level for a hard model.

The nestedness proposition in Jia et al. [1, Proposition 3.2] explains when lower-level point sets are contained in higher-level point sets. For implementation, its practical message is simple: nested univariate rules can reduce the number of distinct points after merging. Non-nested rules can still be used, but the duplicate merge will be weaker and the point count can be larger.

Finally, Jia et al. [1, Theorem 3.2] shows that the unscented transform point set appears as a level-2 sparse-grid member under the source's choice of one-point and three-point univariate rules. Here this theorem is used as a conceptual bridge. It tells a reader that SGQF is not a mysterious new filtering principle; it is a systematic sparse-grid way to build deterministic Gaussian moment points. The selected BayesFilter object remains the fixed cloud (3.68).

Why this can look like a UKF picture. A beginner who has seen a UKF or Unscented Transform diagram should notice that low-level sparse-grid clouds can look familiar: one often sees a center point and symmetric off-center points. That geometric similarity is real and useful for intuition. Both methods are deterministic weighted point clouds used to approximate Gaussian expectations. But the construction rule is different.

What is similar and what is different. At a high level, SGQF clouds and UKF-style sigma-point sets are similar in three ways: both are deterministic weighted point clouds, both are centered around the Gaussian mean, and both are used to approximate Gaussian expectations. But the difference in construction is important. A UKF is usually introduced as one directly specified sigma-point set with its own location and weighting formulas. By contrast, SGQF is built indirectly: choose a one-dimensional rule family, choose sparse-grid level L, determine which tensor rules are active, attach Smolyak coefficients, and merge repeated point contributions.

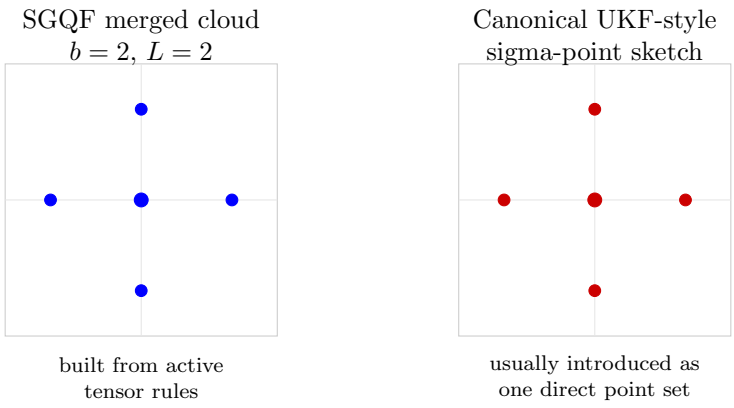

Figure 2: A safe visual analogy. The low-level SGQF cloud and a canonical UKF-style sigma-point sketch can look similar geometrically: both show a center point plus symmetric axis points. The similarity is useful for intuition, but it does not mean the construction is the same.

When the final cloud can look very UKF-like. A further subtlety is worth stating explicitly. If two methods end with the same symmetric axis geometry and both are required to match the same second moments, then their final weights can match. Consider a symmetric axis cloud in dimension d: one center point at the origin with weight w_0, and two axis points $\pm a e_i$ in each coordinate direction, each carrying the same weight w. There are $2d$ off-center points, so total weight 1 requires

$$w_0 + 2dw = 1. \tag{3.71}$$

Matching variance 1 in each coordinate requires

$$2wa^2 = 1. \tag{3.72}$$

Hence

$$w = \frac{1}{2a^2}, \qquad w_0 = 1 - \frac{d}{a^2}. \tag{3.73}$$

So second-moment accuracy alone does not uniquely determine the whole cloud. It only says that once the radius a is chosen, the weights are determined by that radius.

In two dimensions, choosing $a = \sqrt{3}$ gives

$$w = \frac{1}{6}, \qquad w_0 = \frac{1}{3}, \tag{3.74}$$

which matches the merged SGQF level-2 five-point cloud. In three dimensions, choosing $a = \sqrt{3}$ gives

$$w = \frac{1}{6}, \qquad w_0 = 0, \tag{3.75}$$

which matches the low-level SGQF toy cloud after the center cancels. This explains why SGQF can look very UKF-like at low level. But it does *not* mean the constructions are the same; it only means that the final low-level cloud can coincide in special symmetric-axis cases.

3.4.8 A Deterministic Level Ladder Before Failure

For implementation work, "choose a level" is not enough. Use a declared deterministic ladder:

$$L \in \{2, 3, 4, \ldots, L_{\max}\}, \tag{3.76}$$

with a point budget $M_{\max}$, a covariance branch policy, and a validation set of pilot parameter values Θ_{pilot}. For each level, build $\mathcal{C}_{b,L}$, record $M_{b,L}$, and run the construction diagnostics and pilot filtering diagnostics. The first level that satisfies

$$M_{b,L} \le M_{\max}, \tag{3.77}$$

$$\min_{t,\theta\in\Theta_{\text{pilot}}} \lambda_{\min}(S_t(\theta)) \ge \epsilon_S, \tag{3.78}$$

$$\max_{\theta\in\Theta_{\text{pilot}}} \left|\widehat{\ell}_T^{(L)}(\theta) - \widehat{\ell}_T^{(L-1)}(\theta)\right| \le \epsilon_\ell \quad \text{for } L > 2 \tag{3.79}$$

is eligible for a fixed-branch likelihood. If no level satisfies the ladder, FixedSGQF returns a design failure rather than silently using a convenient grid. The numerical constants ϵ_S and ϵ_ℓ are diagnostic thresholds, not mathematical proof parameters. A minimal default table is given later in Section A.3.

3.4.9 Merged Toy Clouds And Duplicate Merging

This section makes the sparse-grid cloud concrete. Start with the 2D case, which is the smallest nontrivial merged example, and then record the matching 3D case explicitly.

3.4.10 Two-dimensional toy cloud

Take $b = 2$ and $L = 2$. Use

$$I_1 : \quad (0,1), \qquad I_2 : \quad \left\{(-\sqrt{3}, 1/6), (0, 2/3), (\sqrt{3}, 1/6)\right\}, \tag{3.80}$$

which is the familiar three-point standard-normal rule at level 2. The band $\mathcal{A}_{2,2}$ contains

$$(1,1), \qquad (1,2), \qquad (2,1), \tag{3.81}$$

with coefficients

$$c_{(1,1)} = -1, \qquad c_{(1,2)} = 1, \qquad c_{(2,1)} = 1. \tag{3.82}$$

Before merging, the node $(0,0)$ appears three times. Its accumulated weight is

$$-1 + \frac{2}{3} + \frac{2}{3} = \frac{1}{3}. \tag{3.83}$$

The four axis nodes each receive weight $1/6$. The merged cloud is

node ξ	accumulated weight	source components
$(0,0)$	$1/3$	$(1,1), (1,2), (2,1)$
$(-\sqrt{3}, 0)$	$1/6$	$(2,1)$
$(\sqrt{3}, 0)$	$1/6$	$(2,1)$
$(0, -\sqrt{3})$	$1/6$	$(1,2)$
$(0, \sqrt{3})$	$1/6$	$(1,2)$

The weights sum to one:

$$\frac{1}{3} + 4 \cdot \frac{1}{6} = 1. \tag{3.84}$$

This toy cloud illustrates two implementation rules that are easy to get wrong. First, duplicate nodes must be accumulated before the cloud is used. Second, signed sparse-grid coefficients are

applied before accumulation; a future level or a different univariate family can produce negative accumulated weights, so covariance matrices require explicit diagnostics.

Algorithmically, this toy example should be read as a dictionary update. Start from an empty map from standardized nodes to accumulated weights. Insert the contribution of each tensor component in (3.81) with its signed coefficient from (3.82). The origin node $(0,0)$ receives three signed contributions and is stored only once with the final accumulated weight from (3.83). The resulting five-node dictionary is exactly the object that is later reused by both the value and gradient paths; no tensor-component provenance is needed after the merge is complete.

3.4.11 Three-dimensional toy cloud

Now carry the same construction through the concrete 3D case from the previous section. Take $b = 3$ and $L = 2$, and reuse the same one-dimensional rules I_1 and I_2 from (3.80). The active band is

$$\mathcal{A}_{3,2} = \{(1,1,1), (1,1,2), (1,2,1), (2,1,1)\}, \tag{3.85}$$

with coefficients

$$c_{(1,1,1)} = -2, \qquad c_{(1,1,2)} = 1, \qquad c_{(1,2,1)} = 1, \qquad c_{(2,1,1)} = 1. \tag{3.86}$$

Before merging, the tensor components contribute:

- $(1,1,1)$: the single point $(0,0,0)$, with signed contribution -2,
- $(1,1,2)$: the three points $(0,0,-\sqrt{3})$, $(0,0,0)$, $(0,0,\sqrt{3})$, with signed weights 1/6, 2/3, 1/6,
- $(1,2,1)$: the three points $(0,-\sqrt{3},0)$, $(0,0,0)$, $(0,\sqrt{3},0)$, with signed weights 1/6, 2/3, 1/6,
- $(2,1,1)$: the three points $(-\sqrt{3},0,0)$, $(0,0,0)$, $(\sqrt{3},0,0)$, with signed weights 1/6, 2/3, 1/6.

The origin $(0,0,0)$ therefore appears in all four tensor components. Its accumulated weight is

$$-2 + \frac{2}{3} + \frac{2}{3} + \frac{2}{3} = 0. \tag{3.87}$$

So the center point cancels completely after duplicate merging. The six axis points each appear once and keep weight 1/6. The merged cloud is

node ξ	accumulated weight	source component
$(\sqrt{3},0,0)$	1/6	$(2,1,1)$
$(-\sqrt{3},0,0)$	1/6	$(2,1,1)$
$(0,\sqrt{3},0)$	1/6	$(1,2,1)$
$(0,-\sqrt{3},0)$	1/6	$(1,2,1)$
$(0,0,\sqrt{3})$	1/6	$(1,1,2)$
$(0,0,-\sqrt{3})$	1/6	$(1,1,2)$

The surviving weights again sum to one:

$$6 \cdot \frac{1}{6} = 1. \tag{3.88}$$

This explicit 3D cloud shows the sparse-grid economy in its final merged form. The full 27-point tensor-product picture is only a comparator. The actual sparse-grid construction at $(b, L) = (3, 2)$

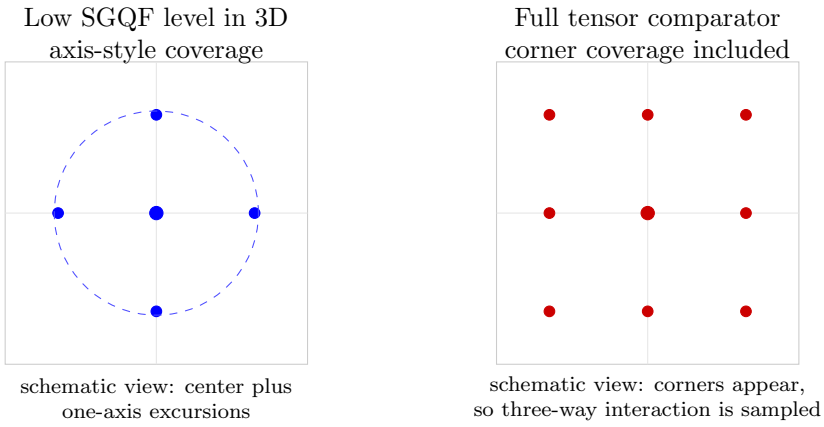

Figure 3: A schematic 3D comparison. A low sparse-grid level keeps center and axis-type structure, while the full tensor comparator also includes corner coverage. For functions driven by genuine three-way products, those corners can matter substantially.

first forms the signed combination of $(1,1,1)$, $(1,1,2)$, $(1,2,1)$, and $(2,1,1)$, producing 10 raw contributions, then 7 merged distinct nodes, and finally 6 stored nodes after zero-weight pruning removes the canceled center. That stored 6-point cloud is the concrete 3D object the later high-dimensional notation is abstracting.

To keep the same test function (3.30) in view, observe again that every surviving axis point has value $F = 3$ and the center point has already canceled. So the merged-cloud sum is just

$$\frac{1}{6}\cdot 3+\frac{1}{6}\cdot 3+\frac{1}{6}\cdot 3+\frac{1}{6}\cdot 3+\frac{1}{6}\cdot 3+\frac{1}{6}\cdot 3=3, \tag{3.89}$$

which matches the exact Gaussian expectation for this low-order quadratic test function. This is the concrete computational sense in which the sparse-grid reduction "works" in the present example: it reduces the point count substantially while still preserving the targeted low-order structure.

4 Filtering Value Path And Worked Example

Fix a standardized sparse-grid cloud

$$\mathcal{C}=\{(\xi^{(r)},w_r)\}_{r=1}^{M}, \qquad \xi^{(r)}\in\mathbb{R}^{n_x}, \quad w_r\in\mathbb{R}. \tag{4.1}$$

The cloud is frozen before likelihood evaluation. The value path has two quadrature stages at each time.

What this section gives the implementer. This section is the authoritative one-step value recursion. A value routine constructed from this note should take the current carried Gaussian, current observation, fixed cloud, and declared model objects as input, then return either an accepted one-step increment with updated (m_t, P_t) or a branch failure record.

4.1 Prediction Stage

Let $P_{t-1}=C_{t-1}C_{t-1}^{\top}$ on the declared square-root branch. Place the previous Gaussian cloud:

$$\chi_{t-1}^{(r)}=m_{t-1}+C_{t-1}\xi^{(r)}. \tag{4.2}$$

Push points through the transition:

$$a_t^{(r)} = f_\theta(\chi_{t-1}^{(r)}). \tag{4.3}$$

The predicted Gaussian is

$$m_t^- = \sum_{r=1}^{M} w_r a_t^{(r)}, \tag{4.4}$$

$$P_t^- = Q_\theta + \sum_{r=1}^{M} w_r (a_t^{(r)} - m_t^-)(a_t^{(r)} - m_t^-)^\top. \tag{4.5}$$

After (4.5), the implementation replaces P_t^- by $(P_t^- + P_t^{-\top})/2$ and checks the declared positive-definite branch. For the differentiable lane in this note, there is no adaptive jitter, floor, clipping, pivot, or eigenvalue repair inside the scalar. If the check fails, the value path returns a veto rather than quietly changing the scalar.

4.2 Observation Stage

Factor $P_t^- = C_t^-(C_t^-)^\top$ on the declared branch and place the predictive cloud:

$$\chi_t^{(r)} = m_t^- + C_t^- \xi^{(r)}. \tag{4.6}$$

Evaluate the observation map:

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}). \tag{4.7}$$

Then compute

$$\bar{z}_t = \sum_{r=1}^{M} w_r z_t^{(r)}, \tag{4.8}$$

$$\Delta_t^{(r)} = z_t^{(r)} - \bar{z}_t, \tag{4.9}$$

$$S_t = R_\theta + \sum_{r=1}^{M} w_r \Delta_t^{(r)} \Delta_t^{(r)\top}, \tag{4.10}$$

$$C_{xz,t} = \sum_{r=1}^{M} w_r (\chi_t^{(r)} - m_t^-) \Delta_t^{(r)\top}. \tag{4.11}$$

The innovation and log likelihood are

$$v_t = y_t - \bar{z}_t, \tag{4.12}$$

$$\widehat{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right\}. \tag{4.13}$$

If S_t is not positive definite on the declared branch, the value path returns a veto. If it passes, the Gaussian projection update is

$$K_t = C_{xz,t} S_t^{-1}, \tag{4.14}$$

$$m_t = m_t^- + K_t v_t, \tag{4.15}$$

$$P_t = P_t^- - K_t S_t K_t^\top. \tag{4.16}$$

Value-path pseudocode summary. The runtime order defined by this section is:

1. symmetrize and factor P_{t-1},
2. place previous-state points and propagate through f_θ,
3. form (m_t^-, P_t^-) and veto if the predictive branch fails,
4. factor P_t^-, place predictive points, and propagate through h_θ,
5. form $\bar{z}_t$, S_t, and $C_{xz,t}$, and veto if the innovation branch fails,
6. form v_t, $\widehat{\ell}_t^{\mathrm{FSGQ}}$, K_t, m_t, and P_t, then veto if the carried covariance branch fails,
7. otherwise return the one-step increment, updated carried Gaussian, and any diagnostics cached for derivative reuse.

This pseudocode summary does not replace the equations above; it states their required execution order.

4.3 Worked One-Step Example With A Numeric Oracle

This section gives one compact example that can be read two ways. First, it is small enough for a chair to teach back without reconstructing the notation from scratch. Second, it gives an implementation engineer a concrete one-step oracle against which a first prototype can be checked.

Use the one-dimensional transition-observation model

$$X_0 \sim \mathcal{N}(m_0, P_0), \qquad m_0 = 0.5, \qquad P_0 = 1, \tag{4.17}$$

$$X_1 = f_\beta(X_0) + \eta_1, \qquad f_\beta(x) = \beta x, \qquad \eta_1 \sim \mathcal{N}(0, Q), \tag{4.18}$$

$$Y_1 = h_\theta(X_1) + \varepsilon_1, \qquad h_\theta(x) = x^2, \qquad \varepsilon_1 \sim \mathcal{N}(0, R), \tag{4.19}$$

with

$$Q = 0.25, \qquad R = 1, \qquad y_1 = 2. \tag{4.20}$$

Differentiate with respect to the single parameter component β. For this chosen component, the transition depends on β but the observation map does not, so

$$\partial_\beta h_\theta(x) = 0. \tag{4.21}$$

Evaluate all quantities at $\beta = 1$, and use the positive scalar Cholesky branch $C(P) = \sqrt{P}$ with no repair. Because the example yields $P_1^- = 5/4 > 0$ and $S_1 = 43/8 > 0$, both the value and derivative lanes stay on that same declared branch. Use the three-point standard-normal rule

$$\mathcal{C}_{1,2} = \left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{4.22}$$

The same three-node cloud is reused without change in both the value and derivative calculations, so the example can be taught back as one declared branch with one declared cloud across both lanes.

Value path. Since $C_0 = \sqrt{P_0} = 1$, the previous-state placed points are

$$\chi_0^{(r)} = m_0 + C_0\xi^{(r)} \in \{0.5 - \sqrt{3},\ 0.5,\ 0.5 + \sqrt{3}\}. \tag{4.23}$$

At $\beta = 1$, the transition images satisfy

$$a_1^{(r)} = f_\beta(\chi_0^{(r)}) = \beta\chi_0^{(r)} = \chi_0^{(r)}. \tag{4.24}$$

The predicted Gaussian moments are therefore

$$m_1^- = \sum_r w_r a_1^{(r)} = 0.5, \qquad P_1^- = Q + \sum_r w_r(a_1^{(r)} - m_1^-)^2 = 1.25 = \frac{5}{4}. \tag{4.25}$$

The declared Cholesky factor is

$$C_1^- = \mathrm{chol}(P_1^-) = \sqrt{1.25} = \frac{\sqrt{5}}{2}. \tag{4.26}$$

Hence the predictive placed points are

$$\chi_1^{(r)} = m_1^- + C_1^-\xi^{(r)} \in \left\{0.5 - \frac{\sqrt{15}}{2},\ 0.5,\ 0.5 + \frac{\sqrt{15}}{2}\right\}. \tag{4.27}$$

Passing those points through $h_\theta(x) = x^2$ gives

$$z_1^{(r)} \in \{2.0635083269,\ 0.25,\ 5.9364916731\}. \tag{4.28}$$

The centered predictive and observation values are

$$d_1^{(r)} = a_1^{(r)} - m_1^- \in \{-\sqrt{3},\ 0,\ \sqrt{3}\}, \tag{4.29}$$

and

$$\Delta_1^{(r)} = z_1^{(r)} - \bar{z}_1 \in \{0.5635083269,\ -1.25,\ 4.4364916731\}. \tag{4.30}$$

The observation moments and Gaussian update objects are

$$\bar{z}_1 = \sum_r w_r z_1^{(r)} = 1.5 = \frac{3}{2}, \tag{4.31}$$

$$S_1 = R + \sum_r w_r(z_1^{(r)} - \bar{z}_1)^2 = 5.375 = \frac{43}{8}, \tag{4.32}$$

$$C_{xz,1} = \sum_r w_r(\chi_1^{(r)} - m_1^-)(z_1^{(r)} - \bar{z}_1) = 1.25 = \frac{5}{4}, \tag{4.33}$$

$$v_1 = y_1 - \bar{z}_1 = 0.5 = \frac{1}{2}, \tag{4.34}$$

$$K_1 = C_{xz,1}S_1^{-1} = \frac{10}{43} \approx 0.2325581395, \tag{4.35}$$

$$m_1 = m_1^- + K_1 v_1 = \frac{53}{86} \approx 0.6162790698, \tag{4.36}$$

$$P_1 = P_1^- - K_1 S_1 K_1^\top = \frac{165}{172} \approx 0.9593023256. \tag{4.37}$$

The one-step deterministic approximate log likelihood is

$$\widehat{\ell}_1^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{\log S_1 + v_1^2 S_1^{-1} + \log(2\pi)\right\} \approx -1.7830736342. \tag{4.38}$$

Same-branch derivative with respect to β. The same cloud and the same positive scalar Cholesky branch are reused in the derivative lane. This example is chosen so that the parameter enters only through the transition, not through the observation map, which is why $\partial_\beta h_\theta(x) = 0$. Because m_0 and P_0 are fixed in β, the previous-state placed points satisfy

$$\dot{\chi}_0^{(r)} = 0. \tag{4.39}$$

From (5.15),

$$\dot{a}_1^{(r)} = D_x f_\beta(\chi_0^{(r)})\dot{\chi}_0^{(r)} + \partial_\beta f_\beta(\chi_0^{(r)}) = \chi_0^{(r)} \in \{0.5 - \sqrt{3},\ 0.5,\ 0.5 + \sqrt{3}\}. \tag{4.40}$$

Hence

$$\dot{m}_1^- = \sum_r w_r \dot{a}_1^{(r)} = m_0 = 0.5, \qquad \dot{d}_1^{(r)} = \dot{a}_1^{(r)} - \dot{m}_1^- \in \{-\sqrt{3},\ 0,\ \sqrt{3}\}, \tag{4.41}$$

and

$$\dot{P}_1^- = \partial_\beta(\beta^2 P_0 + Q) = 2. \tag{4.42}$$

Because $C_1^- = \sqrt{P_1^-}$, the declared Cholesky derivative is

$$\dot{C}_1^- = \frac{\dot{P}_1^-}{2C_1^-} = \frac{2}{\sqrt{5}} \approx 0.8944271910. \tag{4.43}$$

Therefore the predictive point sensitivities are

$$\dot{\chi}_1^{(r)} = \dot{m}_1^- + \dot{C}_1^- \xi^{(r)} \in \{-1.0491933385,\ 0.5,\ 2.0491933385\}. \tag{4.44}$$

Since $h_\theta(x) = x^2$ and $\partial_\beta h_\theta(x) = 0$,

$$\dot{z}_1^{(r)} = 2\chi_1^{(r)}\dot{\chi}_1^{(r)} \in \{3.0143149884,\ 0.5,\ 9.9856850116\}. \tag{4.45}$$

The centered observation sensitivities are

$$\dot{\Delta}_1^{(r)} = \dot{z}_1^{(r)} - \dot{\bar{z}}_1 \in \{0.5143149884,\ -2,\ 7.4856850116\}. \tag{4.46}$$

The derived moment sensitivities are

$$\dot{\bar{z}}_1 = 2.5 = \frac{5}{2}, \qquad \dot{v}_1 = -2.5 = -\frac{5}{2}, \tag{4.47}$$

$$\dot{S}_1 = 14.5 = \frac{29}{2}, \qquad \dot{C}_{xz,1} = 3.25 = \frac{13}{4}. \tag{4.48}$$

Let

$$u_1 = S_1^{-1} v_1 = \frac{4}{43} \approx 0.0930232558. \tag{4.49}$$

The three scalar ingredients in the score formula are

$$\operatorname{tr}(S_1^{-1}\dot{S}_1) = \frac{\dot{S}_1}{S_1} = \frac{116}{43} \approx 2.6976744186, \tag{4.50}$$

$$2\dot{v}_1 u_1 = -\frac{20}{43} \approx -0.4651162791, \tag{4.51}$$

$$-u_1^2 \dot{S}_1 = -\frac{232}{1849} \approx -0.1254732288. \tag{4.52}$$

Then the same-branch one-step score is

$$\dot{\ell}_1^{\mathrm{FSGQ}} = -\frac{1}{2}\left[\frac{\dot{S}_1}{S_1} + 2\dot{v}_1 u_1 - u_1^2 \dot{S}_1\right] = -\frac{1948}{1849} \approx -1.0535424554. \tag{4.53}$$

For later propagation,

$$\dot{K}_1 = \dot{C}_{xz,1} S_1^{-1} - K_1 \dot{S}_1 S_1^{-1} = -\frac{42}{1849} \approx -0.0227149811, \tag{4.54}$$

so

$$\dot{m}_1 = \dot{m}_1^- + \dot{K}_1 v_1 + K_1 \dot{v}_1 = -\frac{343}{3698} \approx -0.0927528394, \tag{4.55}$$

$$\dot{P}_1 = \dot{P}_1^- - \dot{K}_1 S_1 K_1 - K_1 \dot{S}_1 K_1 - K_1 S_1 \dot{K}_1 = \frac{2353}{1849} \approx 1.2725797729. \tag{4.56}$$

The example makes the report's main limitation concrete. Every quantity above is deterministic and internally checkable on the declared branch, yet the observation law $Y_1 = X_1^2 + \varepsilon_1$ still has the same qualitative ability to generate non-Gaussian posterior structure emphasized earlier in the scalar-cell warning. FixedSGQF therefore gives a transparent Gaussian-surrogate value-and-gradient oracle, not a full non-Gaussian posterior representation.

Implementation checklist for this oracle. A first implementation should be able to reproduce, up to declared floating-point tolerance, at least the following quantities from this section: m_1^-, P_1^-, $\bar{z}_1$, S_1, $C_{xz,1}$, K_1, m_1, P_1, $\widehat{\ell}_1^{\mathrm{FSGQ}}$, and $\dot{\ell}_1^{\mathrm{FSGQ}}$. This is the compact end-to-end numerical oracle for both the value path and the derivative path.

The next section now states the exact saved-scalar contract that the worked example has instantiated numerically: which structural choices are frozen, which numerical quantities are recomputed, and what it means for value, gradient, and finite differences to refer to the same declared target.

5 Same-Scalar Branch Contract And Analytical Gradient

The branch record $\mathcal{B}$ freezes both the value-defining structural choices and the derivative-support metadata needed to differentiate the same scalar on that branch. The value scalar itself depends only on the former, while the derivative-support objects matter only when the gradient lane is requested. A fixed-SGQF branch is the tuple

$$\begin{aligned}\mathcal{B} = (&y_{1:T},\ \text{observation preprocessing},\ m_0(\theta), P_0(\theta), \dot{m}_0, \dot{P}_0,\\ &\mathcal{C},\ \text{one-dimensional rules},\ \text{duplicate-merge tolerance},\ \text{zero-weight rule},\ \text{node ordering},\\ &C(P) = \mathrm{chol}(P),\ \text{symmetrize-then-veto rule},\ \epsilon_S, \epsilon_P,\\ &f_\theta, h_\theta, Q_\theta, R_\theta,\ D_x f_\theta, \partial_i f_\theta, D_x h_\theta, \partial_i h_\theta, \dot{Q}_\theta, \dot{R}_\theta).\end{aligned} \tag{5.1}$$

The fixed scalar is

$$\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta; \mathcal{B}) = \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{FSGQ}}(\theta; \mathcal{B}), \tag{5.2}$$

where every term is computed by (4.2)–(4.16). For the value path, only the value-defining structural subset of $\mathcal{B}$ enters this scalar: cloud choice, one-dimensional rules, merge/prune/order policy,

observation preprocessing, initial-condition policy, factor family, and veto thresholds. The derivative-support routines and differentiated initial objects listed inside $\mathcal{B}$ are not extra arguments to the value scalar; they are the metadata needed to differentiate that same declared scalar without changing its branch identity. The same full $\mathcal{B}$ must be used for values, gradients, and finite differences. Changing the grid level, active index set, duplicate-merge tolerance, zero-weight pruning rule, node ordering, Cholesky pivot, observation preprocessing, initial-condition policy, eigenvalue floor, covariance clipping, or point pruning changes the scalar. The derivative formulas below cover the open branch where the symmetrized covariances pass the Cholesky and positive-definite checks without repair. If a jitter, floor, clipping, or pivoting map is desired, it must be declared as a different scalar with its own derivative or stop rule.

Plain-language branch rule. The branch identity freezes structure, not numerical values. When θ changes inside a same-scalar finite-difference check, the moments, gains, and likelihood contributions are recomputed, but they must be recomputed by the same cloud, the same merge/order policy, the same factor family, and the same realized acceptance path through the declared branch logic. This is the operational meaning of "same scalar" for implementation review.

Frozen versus variable objects on a saved branch. The branch record $\mathcal{B}$ freezes structural choices: the cloud, merge/prune/order policy, factor family, veto thresholds, preprocessing policy, and initial-condition policy. It does *not* freeze the numerical values of the moments or filters. Within a fixed branch,

$$\begin{aligned} \text{frozen structural objects:} &\quad \mathcal{C},\ \tau_{\text{merge}},\ \tau_{\text{zero}},\ \text{ordering},\ C(\cdot),\ \epsilon_S, \epsilon_P, \\ \text{parameter-dependent numerical objects:} &\quad m_t, P_t, C_t, m_t^-, P_t^-, C_t^-, \bar{z}_t, S_t, C_{xz,t}, K_t, \widehat{\ell}_t^{\text{FSGQ}}. \end{aligned} \tag{5.3}$$

Thus same-scalar finite differences recompute the parameter-dependent numerical objects by the same declared branch rule; they do not reuse old numerical values. In particular, $m_0(\theta \pm he_i)$ and $P_0(\theta \pm he_i)$ are recomputed by the same initial-condition policy; only the policy is frozen, not the numerical initial values. What must remain unchanged is the structural route by which those values are produced.

Operational branch identity for finite differences. For implementation review, define the branch-identity record

$$\mathfrak{B} = (\mathcal{C},\ \tau_{\text{merge}},\ \tau_{\text{zero}},\ \text{ordering, preprocessing policy, initial-condition policy, symmetrize-then-veto rule, } C(P) = \tag{5.4}$$

The accepted/failure stage-time pattern means the realized branch path through each likelihood contribution: at every time step it records whether the predictive covariance check, innovation covariance check, and carried-covariance check were accepted, and if not, which stage failed first. A central, forward, or backward finite-difference row is branch-valid only if both perturbed evaluations share the same saved structural metadata and the same realized acceptance path as the base evaluation through every likelihood contribution included in the comparison. Equal factor family but different first failure location counts as branch mismatch. Equal first failure location but different accepted-prefix pattern also counts as branch mismatch. If either side changes the accepted stage-time pattern, triggers a different veto location, or uses a different cloud/merge/order/preprocessing/initial-condition specification, the row is branch-invalid and is not interpreted as evidence about the derivative.

5.0.1 Analytical Gradient Of The Fixed Scalar

The value path has now fixed the declared scalar and the branch on which that scalar is evaluated. The next task is not to introduce a new target, but to differentiate that same declared target in a way that can still be checked by finite differences. Differentiate with respect to parameter component θ_i. A dot denotes ∂_i, for example $\dot{m}_t = \partial_i m_t$. The data y_t are recorded and do not depend on θ.

Why this derivative is operationally delicate. The one-step scalar at time t depends on objects constructed earlier in the filter: the previous carried Gaussian, the covariance factor used to place the cloud, the nonlinear transition and observation images, the innovation covariance, and the posterior update. The derivative must therefore propagate through the entire recursion, and it is meaningful only when the value and derivative paths stay on the same declared branch.

Current-score objects versus propagation objects. For the current likelihood contribution, the derivative closes once $\dot{v}_t$ and $\dot{S}_t$ are known. The derivative $\dot{C}_{xz,t}$ is not needed to close the current score, but it is needed to form $\dot{K}_t$ and hence to propagate $\dot{m}_t$ and $\dot{P}_t$ to the next time step. A correct implementation must keep these two roles separate.

5.0.2 The Story Of The Derivative

The derivative is long because the scalar at time t depends on objects created earlier in the filter. This section therefore proceeds in six explicit stages: factor derivative, point sensitivities, predictive moment sensitivities, observation moment sensitivities, innovation score, and posterior sensitivity propagation. The chain is

$$\begin{aligned}\theta &\longmapsto (m_{t-1}, P_{t-1}) \longmapsto C_{t-1} \longmapsto \chi_{t-1}^{(r)} \longmapsto a_t^{(r)} \longmapsto (m_t^-, P_t^-) \longmapsto C_t^- \\ &\longmapsto \chi_t^{(r)} \longmapsto z_t^{(r)} \longmapsto (\bar{z}_t, S_t, C_{xz,t}) \longmapsto (v_t, K_t, \ell_t^{\mathrm{FSGQ}}, m_t, P_t).\end{aligned} \tag{5.5}$$

The sparse-grid nodes and weights do not appear with dots in this chain. They are saved branch objects. Their job is to say where the deterministic moment sum is evaluated, not to move when θ moves. The Gaussian factors C_{t-1} and C_t^-, however, do move with θ, because they are functions of P_{t-1} and P_t^-. That is why the factor derivative is part of the scalar contract. For readers tracking the derivation operationally, the six stages below can be read as a ledger of which object is introduced, what it depends on, and whether it closes the present score or only prepares the next step.

$$\begin{aligned}\text{previous factor stage} &\Longrightarrow \dot{C}_{t-1}, \\ \text{prediction stage} &\Longrightarrow \dot{\chi}_{t-1}^{(r)}, \dot{a}_t^{(r)}, \dot{m}_t^-, \dot{P}_t^-, \\ \text{predictive factor stage} &\Longrightarrow \dot{C}_t^-, \\ \text{observation stage} &\Longrightarrow \dot{\chi}_t^{(r)}, \dot{z}_t^{(r)}, \dot{\bar{z}}_t, \dot{v}_t, \dot{S}_t, \dot{C}_{xz,t}, \\ \text{score stage} &\Longrightarrow \dot{\ell}_t^{\mathrm{FSGQ}}, \\ \text{propagation stage} &\Longrightarrow \dot{K}_t, \dot{m}_t, \dot{P}_t.\end{aligned} \tag{5.6}$$

The value path computes the fixed-branch one-step scalar

$$\ell_t^{\mathrm{FSGQ}} = -\frac{1}{2}\{\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\}.$$

The gradient path differentiates that same line. It is helpful to separate the one-step derivative into the three pieces that will later reappear in the score formula:

$$\partial_i \widehat{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2}\,\partial_i \log\det S_t - \frac{1}{2}\,\partial_i (v_t^\top S_t^{-1} v_t) - \frac{1}{2}\,\partial_i (n_y \log 2\pi). \tag{5.7}$$

Since the last term is zero, the current-period score is controlled entirely by how S_t and v_t move. That is why the gradient path needs only two objects for the current likelihood contribution:

$$\dot{v}_t, \qquad \dot{S}_t.$$

The cross-covariance derivative $\dot{C}_{xz,t}$ is not needed for $\dot{\widehat{\ell}}_t$. It is needed because $C_{xz,t}$ forms the gain K_t, and K_t updates m_t and P_t. Those updated sensitivities become the input to the next time step. Thus the derivative has two roles:

$$\text{score role:} \quad (\dot{v}_t, \dot{S}_t) \mapsto \dot{\widehat{\ell}}_t^{\mathrm{FSGQ}}, \tag{5.8}$$

$$\text{propagation role:} \quad (\dot{C}_{xz,t}, \dot{S}_t, \dot{v}_t) \mapsto (\dot{m}_t, \dot{P}_t). \tag{5.9}$$

Confusing these roles is a common implementation mistake. A program can appear to compute a correct one-step score while still propagating the wrong next-step derivative.

object	role	why it matters
$\dot{v}_t, \dot{S}_t$	current score	sufficient for the present innovation score in (5.28)
$\dot{C}_{xz,t}$	propagation only	needed because K_t depends on $C_{xz,t}$, even though the current score does not
$\dot{K}_t$	propagation only	transmits observation sensitivity into the posterior update
$\dot{m}_t, \dot{P}_t$	next-step state	become the differentiated inputs to the next prediction stage and the next factor derivative

There are four places where the derivative can stop:

1. P_{t-1} or P_t^- is not on the saved factor branch.
2. S_t is not positive definite.
3. A finite-difference perturbation changes the saved branch, grid, merge, or preprocessing policy.
4. A model derivative routine $D_x f_\theta$, $\partial_i f_\theta$, $D_x h_\theta$, $\partial_i h_\theta$, $\dot{Q}_\theta$, or $\dot{R}_\theta$ is missing or inconsistent with the value routine.

On those events the note does not patch the scalar by hidden repair. It returns a branch failure. This narrowness is exactly what lets the analytical gradient be checked by finite differences.

5.0.3 Square-Root Branch

This first stage differentiates the factor map itself. It is needed before any point sensitivity can be written down, because the placed cloud points depend on C_{t-1} and C_t^-. If this stage is wrong, every later derivative object is wrong even if the score algebra itself is correct. For the default unpivoted Cholesky branch, $P = LL^\top$ with positive diagonal and $C(P) = L$. The implementation convention

is: always use the lower triangular Cholesky factor returned by the declared factor map, never a symmetric square root or pivoted alternative inside this scalar. Given symmetric $\dot{P}$, set

$$A = L^{-1}\dot{P}L^{-\top}. \tag{5.10}$$

Define the lower-triangular operator

$$\Phi(A)_{jk} = \begin{cases} A_{jk}, & j > k, \\ A_{jj}/2, & j = k, \\ 0, & j < k. \end{cases} \tag{5.11}$$

Then

$$\dot{L} = L\Phi(A). \tag{5.12}$$

Equation (5.12) is the governing implementation recipe for every factor derivative in this report. In particular, whenever the value path computes a covariance factor by $C_t = \mathrm{chol}(P_t)$ or $C_t^- = \mathrm{chol}(P_t^-)$, the gradient path computes the matching $\dot{C}_t$ or $\dot{C}_t^-$ by applying (5.10)–(5.12) to the corresponding $\dot{P}_t$ or $\dot{P}_t^-$. No alternative factor derivative is allowed inside the same scalar. Indeed, $\Phi(A)+\Phi(A)^\top = A$, so

$$\dot{L}L^\top + L\dot{L}^\top = L\{\Phi(A) + \Phi(A)^\top\}L^\top = \dot{P}. \tag{5.13}$$

If a different square-root map is used, the implementation must provide $\dot{C} = DC(P)[\dot{P}]$ for the same branch. That alternative is outside the scope of this report: every formula and algorithm below assumes the declared lower-triangular Cholesky branch $C(P) = \mathrm{chol}(P)$.

5.0.4 Prediction Sensitivities

This stage moves the derivative from the previous carried Gaussian into the prediction cloud and the predictive moments. Its outputs, $\dot{m}_t^-$ and $\dot{P}_t^-$, are not yet enough for the current likelihood score, but they are necessary because both the next observation cloud and the next factor derivative depend on them. From (4.2),

$$\dot{\chi}_{t-1}^{(r)} = \dot{m}_{t-1} + \dot{C}_{t-1}\xi^{(r)}. \tag{5.14}$$

From (4.3),

$$\dot{a}_t^{(r)} = D_x f_\theta(\chi_{t-1}^{(r)})\dot{\chi}_{t-1}^{(r)} + \partial_i f_\theta(\chi_{t-1}^{(r)}). \tag{5.15}$$

The predicted mean derivative is

$$\dot{m}_t^- = \sum_{r=1}^{M} w_r \dot{a}_t^{(r)}. \tag{5.16}$$

Let

$$d_t^{(r)} = a_t^{(r)} - m_t^-, \qquad \dot{d}_t^{(r)} = \dot{a}_t^{(r)} - \dot{m}_t^-. \tag{5.17}$$

Then

$$\dot{P}_t^- = \dot{Q}_\theta + \sum_{r=1}^{M} w_r \left\{ \dot{d}_t^{(r)} d_t^{(r)\top} + d_t^{(r)} \dot{d}_t^{(r)\top} \right\}. \tag{5.18}$$

5.0.5 Observation Sensitivities

This stage converts predictive-state sensitivity into observation-side sensitivity. It is here that the derivative splits into the two objects needed for the current score, $\dot{v}_t$ and $\dot{S}_t$, and the extra object needed only for propagation, $\dot{C}_{xz,t}$. The logic is:

$$\dot{\chi}_t^{(r)} \longmapsto \dot{z}_t^{(r)} \longmapsto (\dot{\bar{z}}_t, \dot{v}_t, \dot{\Delta}_t^{(r)}) \longmapsto (\dot{S}_t, \dot{C}_{xz,t}). \tag{5.19}$$

Differentiate (4.6):

$$\dot{\chi}_t^{(r)} = \dot{m}_t^- + \dot{C}_t^- \xi^{(r)}. \tag{5.20}$$

Then

$$\dot{z}_t^{(r)} = D_x h_\theta(\chi_t^{(r)}) \dot{\chi}_t^{(r)} + \partial_i h_\theta(\chi_t^{(r)}). \tag{5.21}$$

The observation mean, residual, and centered residual derivatives are

$$\dot{\bar{z}}_t = \sum_{r=1}^{M} w_r \dot{z}_t^{(r)}, \tag{5.22}$$

$$\dot{v}_t = -\dot{\bar{z}}_t, \tag{5.23}$$

$$\dot{\Delta}_t^{(r)} = \dot{z}_t^{(r)} - \dot{\bar{z}}_t. \tag{5.24}$$

Differentiate S_t and $C_{xz,t}$:

$$\dot{S}_t = \dot{R}_\theta + \sum_{r=1}^{M} w_r \left\{ \dot{\Delta}_t^{(r)} \Delta_t^{(r)\top} + \Delta_t^{(r)} \dot{\Delta}_t^{(r)\top} \right\}, \tag{5.25}$$

$$\dot{C}_{xz,t} = \sum_{r=1}^{M} w_r \left\{ (\dot{\chi}_t^{(r)} - \dot{m}_t^-) \Delta_t^{(r)\top} + (\chi_t^{(r)} - m_t^-) \dot{\Delta}_t^{(r)\top} \right\}. \tag{5.26}$$

5.0.6 Innovation Score

At this point the current-period score problem is local. Once $\dot{v}_t$, $\dot{S}_t$, and the innovation solve variable are known, the pointwise cloud no longer appears explicitly in the one-step score. The proposition below is therefore the score-stage payoff of all earlier derivative work.

Proposition 2 (Fixed Gaussian innovation score)**.** *Assume S_t is positive definite and the active branch is differentiable. Let u_t solve*

$$S_t u_t = v_t. \tag{5.27}$$

Then the derivative of (4.13) is

$$\dot{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2} \left[\operatorname{tr}(S_t^{-1} \dot{S}_t) + 2\dot{v}_t^\top u_t - u_t^\top \dot{S}_t u_t \right]. \tag{5.28}$$

Proof. Use

$$\mathrm{d} \log\det S = \operatorname{tr}(S^{-1}\, \mathrm{d}S) \tag{5.29}$$

and

$$\mathrm{d}S^{-1} = -S^{-1}(\,\mathrm{d}S)S^{-1}. \tag{5.30}$$

Therefore, with $u = S^{-1}v$,

$$\mathrm{d}(v^\top S^{-1} v) = 2(\,\mathrm{d}v)^\top u - u^\top(\,\mathrm{d}S)u. \tag{5.31}$$

Substitute $\mathrm{d}S = \dot{S}_t$ and $\mathrm{d}v = \dot{v}_t$ into (**??**). □

5.0.7 Posterior Sensitivity Propagation

If the note stopped at the innovation score, it would describe only how to differentiate one isolated likelihood contribution. The filter, however, continues in time. This stage therefore propagates the current observation-side sensitivity back into the carried Gaussian that seeds the next step. The gradient must also propagate $\dot{m}_t$ and $\dot{P}_t$ into the next time step. The implementation order is:

$$(\dot{C}_{xz,t}, \dot{S}_t, \dot{v}_t) \longmapsto \dot{K}_t \longmapsto (\dot{m}_t, \dot{P}_t) \longmapsto \dot{C}_t \quad \text{at the next step through} \quad DC(P_t)[\dot{P}_t]. \tag{5.32}$$

Differentiate $K_t = C_{xz,t} S_t^{-1}$:

$$\dot{K}_t = \dot{C}_{xz,t} S_t^{-1} - K_t \dot{S}_t S_t^{-1}. \tag{5.33}$$

Then

$$\dot{m}_t = \dot{m}_t^- + \dot{K}_t v_t + K_t \dot{v}_t, \tag{5.34}$$

$$\dot{P}_t = \dot{P}_t^- - \dot{K}_t S_t K_t^\top - K_t \dot{S}_t K_t^\top - K_t S_t \dot{K}_t^\top. \tag{5.35}$$

Equations (5.14)–(5.35) define the value-and-gradient recursion for the fixed scalar (5.2). At this point the reader should retain one structural summary: $\dot{v}_t$ and $\dot{S}_t$ close the current score, while $\dot{C}_{xz,t}, \dot{K}_t, \dot{m}_t, \dot{P}_t$ exist because the filter must carry its differentiated state forward in time.

5.1 One Boxed Mathematical Algorithm

FixedSGQF value and gradient for one fixed parameter component on one saved branch. The dot notation in this box means differentiation with respect to one declared parameter component θ_i. In particular, $\dot{m}_t, \dot{P}_t, \dot{C}_t^-, \dot{S}_t, \dot{C}_{xz,t}$, and $\dot{\widehat{\ell}}_t^{\mathrm{FSGQ}}$ in this box are the one-component versions of the indexed quantities $\dot{m}_t^{(i)}, \dot{P}_t^{(i)}, \dot{C}_t^{-(i)}, \dot{S}_t^{(i)}, \dot{C}_{xz,t}^{(i)}$, and $\partial_i \widehat{\ell}_t^{\mathrm{FSGQ}}$ used in the full algorithm. This boxed routine is a compact summary of the full implementation-order algorithm in Section B; when the two presentations differ in brevity, the longer end-to-end algorithm is the authoritative execution order.

1. Choose dimension $b = n_x$, level L, one-dimensional rules I_ℓ, merge tolerance, zero-weight threshold, and deterministic node ordering. Build and save the cloud $\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^M$ by (3.47)–(3.68), including duplicate merge, near-zero pruning, and final deterministic sorting. These cloud-construction choices are part of the saved branch record and therefore part of the declared scalar.

2. Save $y_{1:T}$, observation preprocessing, $m_0(\theta), P_0(\theta)$, the one-component sensitivity policy for θ_i, the full model objects $f_\theta, h_\theta, Q_\theta, R_\theta$, the declared lower-triangular Cholesky factor map $C(P) = \mathrm{chol}(P)$, the symmetrize-then-veto rule, the eigenvalue thresholds (A.8), and model derivative routines. Together with the cloud-construction choices in Step 1, these objects form the saved branch record for the declared scalar.

3. Initialize $m_0, P_0, \dot{m}_0, \dot{P}_0$, the running scalar $\widehat{\ell}_T^{\mathrm{FSGQ}} \leftarrow 0$, and the running derivative $G_i \leftarrow 0$, where $G_i = \partial_i \widehat{\ell}_T^{\mathrm{FSGQ}}$ is the i-th component of the full score vector.

4. For $t = 1, \ldots, T$, symmetrize P_{t-1}, then factor $P_{t-1} = C_{t-1} C_{t-1}^\top$ on the saved branch. For this fixed parameter component θ_i, compute $\dot{C}_{t-1} = DC(P_{t-1})[\dot{P}_{t-1}]$, and then run the prediction value equations (4.2)–(4.5) and prediction sensitivity equations (5.14)–(5.18).

5. If the symmetrized predictive covariance P_t^- fails the declared Cholesky or positive-definite test in (A.8), stop and report a predictive-branch failure.

6. Factor $P_t^- = C_t^- (C_t^-)^\top$ on the saved branch, compute $\dot{C}_t^- = DC(P_t^-)[\dot{P}_t^-]$, and then run the observation value equations (4.6)–(4.11), together with the observation sensitivity equations (5.20)–(5.26).

7. Symmetrize the innovation covariance S_t and, if it fails the declared positive-definite branch test in (A.8), stop and report an innovation-branch failure.

8. Only on that accepted innovation branch, form the one-step increment $\widehat{\ell}_t^{\mathrm{FSGQ}}$ from (4.13), solve $S_t u_t = v_t$, and then continue with score accumulation and the Gaussian projection update.

9. Update $m_t, P_t, \dot{m}_t, \dot{P}_t$ using (4.14)–(4.16) and (5.33)–(5.35) for this fixed component θ_i. Then apply the same symmetrize-then-veto rule to the updated carried covariance: replace P_t by $(P_t + P_t^\top)/2$, and if the symmetrized matrix is not on the declared positive-definite factor branch, stop and report a carried-covariance failure.

6 Verification And Validation

The finite-difference check must call the same value routine with the same saved branch $\mathcal{B}$. For parameter component i, use

$$D_i(h) = \frac{\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta + he_i; \mathcal{B}) - \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta - he_i; \mathcal{B})}{2h}. \tag{6.1}$$

Use the step ladder

$$h \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\} \max\{1, |\theta_i|\}. \tag{6.2}$$

For every h, the $+h$ and $-h$ evaluations must reuse the same saved structural branch record $\mathfrak{B}$, that is, the implementation-level branch-identity record induced by the saved branch context $\mathcal{B}$: the same cloud, one-dimensional rules, preprocessing policy, initial-condition policy, declared Cholesky factor branch, merge tolerance, zero-weight rule, node ordering, and symmetrize-then-veto rule. The numerical initial values $m_0(\theta \pm he_i), P_0(\theta \pm he_i)$ are recomputed by that same policy. Each evaluation also returns its realized accepted/failure stage-time pattern. A row is branch-valid only if both perturbations match the base evaluation in that full branch-identity record and in the realized accepted/failure stage-time pattern through every likelihood contribution included in the scalar comparison. Equal factor family but different first failure location counts as branch mismatch. Equal first failure location but different accepted-prefix pattern also counts as branch mismatch. If either side hits a different branch or returns a veto, that row is marked branch-invalid and is not interpreted as a derivative check. For valid rows, compare $D_i(h)$ against the analytical score G_i by

$$e_i(h) = \frac{|D_i(h) - G_i|}{\max\{1, |D_i(h)|, |G_i|\}}. \tag{6.3}$$

The diagnostic passes when at least two consecutive valid rows have $e_i(h) \le 10^{-5}$ and the smaller-step row is no worse than ten times the larger-step row. This is a numerical diagnostic, not a theorem: failure means one of three things is likely. The derivative formula is wrong, the value and gradient paths are not using the same scalar, or the branch is too nonsmooth or ill-conditioned at that point.

Implementation-facing FD pseudocode summary. A routine of the form `same_scalar_fd_check(`$i, h_{\mathrm{ladder}}, \theta,$ `?` should:

1. evaluate the value path at $\theta \pm he_i$ using the same saved structural branch metadata,
2. mark a row branch-invalid if either side changes the structural branch or the realized accepted/failure stage-time pattern,
3. compute the central difference only for branch-valid rows,
4. compare that central difference against the analytical score with (6.3), and
5. report whether two consecutive valid rows satisfy the declared tolerance rule.

This summary is the operational finite-difference contract for an implementation; the equations above remain the authoritative mathematical definition.

A scalar numerical trace. Let $X \sim \mathcal{N}(0,1)$, $h_\theta(x) = \theta x$, $R = 1$, and $y = 2$. Any fixed Gaussian rule with exact second moment gives

$$\bar{z} = 0, \qquad S(\theta) = 1 + \theta^2, \qquad \widehat{\ell}(\theta) = -\frac{1}{2}\left\{\log(1+\theta^2) + \frac{4}{1+\theta^2} + \log(2\pi)\right\}. \tag{6.4}$$

At $\theta = 1$,

$$G = -\frac{1}{2}\left\{\frac{2}{2} - \frac{4 \cdot 2}{2^2}\right\} = 0.5. \tag{6.5}$$

Central differences give:

h	$D(h)$	G	$\lvert D(h) - G\rvert$
10^{-1}	0.5008108250	0.5	8.10825×10^{-4}
10^{-2}	0.5000083311	0.5	8.33108×10^{-6}
10^{-3}	0.5000000833	0.5	8.33331×10^{-8}
10^{-4}	0.5000000008	0.5	8.33722×10^{-10}

A cloud-sensitive nonlinear trace. The previous trace tests the innovation-score algebra. To exercise the fixed cloud, use the two-dimensional level-2 cloud in Section 3.4.9 and the nonlinear observation

$$h_\theta(x_1, x_2) = \theta(x_1^2 + x_2^2), \qquad X \sim \mathcal{N}(0, I_2), \qquad R = 1, \qquad y = 3. \tag{6.6}$$

For the merged cloud in Section 3.4.9,

$$\sum_r w_r(x_{1,r}^2 + x_{2,r}^2) = 2, \qquad \sum_r w_r(x_{1,r}^2 + x_{2,r}^2)^2 = 6. \tag{6.7}$$

Therefore

$$\bar{z}(\theta) = 2\theta, \qquad S(\theta) = 1 + 2\theta^2, \qquad v(\theta) = 3 - 2\theta, \tag{6.8}$$

if the parameter scales the whole nonlinear observation linearly. To exercise the parameter through placed nonlinear values more sharply, set instead

$$h_\theta(x_1, x_2) = \theta^2(x_1^2 + x_2^2), \tag{6.9}$$

which gives

$$\bar{z}(\theta) = 2\theta^2, \qquad S(\theta) = 1 + 2\theta^4, \qquad v(\theta) = 3 - 2\theta^2. \tag{6.10}$$

At $\theta = 1$, the score from (5.28) is

$$G = -\frac{1}{2}\left[\frac{8}{3} + 2(-4)\frac{1}{3} - \frac{1}{3^2}8\right] = \frac{4}{9}. \tag{6.11}$$

Central differences for the saved cloud scalar give:

h	$D(h)$	G	$\lvert D(h) - G\rvert$
10^{-1}	0.5200303321	0.4444444444	7.55859×10^{-2}
10^{-2}	0.4452146703	0.4444444444	7.70226×10^{-4}
10^{-3}	0.4444521481	0.4444444444	7.70369×10^{-6}
10^{-4}	0.4444445215	0.4444444444	7.70368×10^{-8}

This example depends on the merged sparse-grid fourth moment in (6.7). A duplicate-merge bug or a different saved cloud changes $S(\theta)$, G, and the finite-difference table.

6.0.1 Diagnostics, Accuracy, Memory, And Performance Tests

Fixed SGQF should be tested as an approximate deterministic likelihood, not as an exact posterior method. The minimum validation suite is:

1. **Construction checks.** Verify duplicate-node accumulation, weight totals, signed-weight counts, and deterministic cloud reproduction for small (b, L) cases such as Section 3.4.9.
2. **Linear-Gaussian recovery.** For affine f_θ, h_θ, the Gaussian projection recursion should match the Kalman filter up to quadrature and floating-point tolerance.
3. **Quadratic scalar cell.** Use (6.14) to show both the moment accuracy and the posterior-shape limitation.
4. **Same-scalar finite differences.** Run (6.1) for representative parameters and step sizes.
5. **Signed-weight stability.** Record the minimum eigenvalues of symmetrized P_t^-, S_t, and P_t, plus any branch veto.
6. **Memory and runtime scaling.** Record $M_{b,L}$, model evaluations, wall time, and peak memory as b, L, T change.
7. **Accuracy ladder.** Compare fixed SGQF against tensor-product GHQ in small dimension, Kalman truth in linear-Gaussian models, and particle or TT diagnostics in nonlinear models where those references are available.

Passing these checks supports the statement that the implementation evaluates and differentiates the declared approximate scalar. It does not prove exact posterior accuracy.

6.0.2 Mathematical Test Models

The validation suite should use models that expose different failure modes. The following models are mathematical specifications for future code tests. They are ordered to mirror the report's argumentative burden: first recover exact or near-exact reference behavior where possible, then expose the Gaussian-surrogate limitation, then test the cloud-specific and high-dimensional claims that later feed into the method-selection argument of Section 8.

Model A: linear-Gaussian recovery. Let

$$X_t = A_\theta X_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_\theta), \tag{6.12}$$

$$Y_t = H_\theta X_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_\theta). \tag{6.13}$$

The Kalman filter gives the exact likelihood and exact filtering moments. A correct FixedSGQF implementation should match the Kalman mean, covariance, and innovation scalar up to floating-point tolerance because all required moments are at most quadratic in the Gaussian coordinate. This test checks moment assembly, covariance factors, likelihood algebra, and derivative propagation, but it does not stress nonlinear quadrature accuracy.

$$X \sim \mathcal{N}(0, P), \qquad Y = X^2 + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, R). \tag{6.14}$$

Model B: scalar quadratic observation. Use

$$X_t = \rho X_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, q), \tag{6.15}$$

$$Y_t = X_t^2 + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, r). \tag{6.16}$$

This model is small enough to compare against dense numerical integration. For small r and positive observations, the exact posterior can have two lobes. The test should report both moment agreement and posterior-shape failure. It is the guardrail against accidentally describing FixedSGQF as an exact nonlinear posterior filter.

Model C: cloud-sensitive two-dimensional nonlinear observation. Let

$$X_t = \rho X_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, qI_2), \tag{6.17}$$

$$Y_t = \theta^2(X_{t,1}^2 + X_{t,2}^2) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, r). \tag{6.18}$$

The two-dimensional level-2 cloud in Section 3.4.9 gives the explicit moments in (6.7). This test catches duplicate-node and signed-weight errors because the fourth moment changes when the cloud is assembled incorrectly.

Model D: block-local high-dimensional nonlinear model. For $b = md$, partition $x \in \mathbb{R}^b$ into m blocks $x_{[j]} \in \mathbb{R}^d$. Let $\alpha_j \in \mathbb{R}^d$, $\beta_j, \gamma_j \in \mathbb{R}$, and $B_j \in \mathbb{R}^{d\times d}$. Use the state-space model

$$X_t = A_\theta X_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_\theta), \tag{6.19}$$

$$Y_{t,j} = \alpha_j^\top x_{t,[j]} + \beta_j \|x_{t,[j]}\|^2 + \gamma_j x_{t,[j]}^\top B_j x_{t,[j+1]} + \varepsilon_{t,j}, \tag{6.20}$$

with $j = 1, \dots, m$, $x_{[m+1]} = x_{[1]}$, and observation-noise vector $\varepsilon_t = (\varepsilon_{t,1}, \dots, \varepsilon_{t,m})^\top \sim \mathcal{N}(0, R_\theta)$ independent of the process noise and independent across time in the intended state-space-model sense. This model has controlled low-order interactions. It is designed to test whether moderate sparse-grid levels exploit interaction sparsity and whether point count, memory, and runtime grow as expected. Diagnostics should report $b, L, M_{b,L}, T$, model evaluations per step, peak memory, wall time, and branch failures.

Model E: deliberately hard all-coordinate interaction. Use the same state-side validation template as Model D, but pair it with a deliberately all-coordinate observation. Let

$$X_t = A_\theta X_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_\theta), \tag{6.21}$$

$$Y_t = \delta \prod_{j=1}^{b} (1 + \alpha X_{t,j}) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, r), \tag{6.22}$$

with η_t independent across time and ε_t independent of the latent state conditional on the model. This model is not expected to be easy for a moderate sparse grid. It probes the failure mode where high-order coordinate interactions matter. A good validation report should not hide this case; it should show when the level ladder or covariance diagnostics reject the fixed grid.

Model F: Jia–Xin–Cheng orbit-style benchmark. Jia, Xin, and Cheng evaluate SGQF on an orbit estimation problem [1, Section 4]. This note does not reproduce their full experiment, but a future implementation should include an orbit-style benchmark because it is the source paper's main empirical setting. The test should record the state dimension, observation geometry, noise covariances, chosen sparse-grid level, point count, RMSE, likelihood diagnostics, and finite-difference branch status. This benchmark supports source-faithful comparison; it is not a theorem of general performance.

6.0.3 Large-Scale Report Template

For each large-scale run, record

$$\begin{aligned}\mathcal{R} = (&b, L, M_{b,L}, T, \text{number of parameters}, \text{random seed},\\ &\text{model evaluations per step}, \text{peak memory}, \text{wall time},\\ &\min_t \lambda_{\min}(P_t^-), \min_t \lambda_{\min}(S_t), \min_t \lambda_{\min}(P_t),\\ &\widehat{\ell}_T, \|\nabla \widehat{\ell}_T\|, \text{finite-difference table}, \text{branch failures}).\end{aligned} \tag{6.23}$$

Accuracy claims should be made only against a named comparator:

$$\text{error} = \begin{cases} |\widehat{\ell}_T - \ell_T^{\text{Kalman}}|, & \text{linear-Gaussian test},\\ |\widehat{\ell}_T - \ell_T^{\text{dense}}|, & \text{small dense quadrature test},\\ \text{RMSE or posterior diagnostic against PF/TT reference}, & \text{nonlinear large test}. \end{cases} \tag{6.24}$$

Memory and performance do not rescue a failed accuracy or branch diagnostic. They are interpreted after the veto checks. These validation objects therefore set the stage for the final two interpretive tasks of the report: first, to say what adaptive sparse-grid selection can still contribute once same-scalar requirements are imposed, and second, to decide how FixedSGQF should be judged against neighboring algorithmic lanes.

7 Adaptive Sparse Grids As Grid Design

Singh et al. [2] choose sparse-grid indices adaptively by using admissible index sets, local error indicators, active and old index sets, a global error estimate, and a tolerance. That is useful for designing a cloud before inference. For HMC, however, the live target cannot silently change its grid as θ moves. The safe interpretation is

$$\text{adaptive pilot chooses a cloud} \implies \text{freeze the cloud} \implies \text{run FixedSGQF}. \tag{7.1}$$

If the adaptive index set is rerun inside the likelihood, the value path is a piecewise-defined algorithm with discrete changes. The formulas in Section 5.0.1 do not differentiate those changes. This is why adaptive sparse-grid selection enters the note as an upstream design aid rather than as the selected differentiable likelihood lane. With that restriction now explicit, the report can finally compare FixedSGQF against neighboring methods on the exact criteria that matter for selection.

8 Relation To Neighboring High-Dimensional Filtering Proposals

The previous sections have fixed the mathematical object this note is trying to carry: one Gaussian surrogate (m_t, P_t) together with the deterministic same-scalar likelihood

$$\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta) = \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{FSGQ}}(\theta). \tag{8.1}$$

This section is therefore not a general survey. It asks a narrower mathematical question: among nearby approximate filtering lanes, which constructions define a usable deterministic surrogate scalar, which preserve an analytical derivative of that same scalar on a fixed branch, and which avoid the dense-growth burden that would make the lane unusable in the intended high-dimensional regime?

Quick comparison table.

method	carried object	moment engine	same-scalar derivative lane	main limitation
EKF	one Gaussian	local linearization	yes, for the linearized scalar	weak nonlinear moment fidelity
UKF/CKF	one Gaussian	fixed low-order deterministic points	yes	lower-order deterministic rule
Dense GHQF	one Gaussian	dense tensor-product Gaussian quadrature	yes	exponential point growth
FixedSGQF	one Gaussian	fixed sparse-grid quadrature	yes	still one Gaussian; signed-weight branch sensitivity
Adaptive sparse grid	one Gaussian	parameter-dependent active grid	only if frozen first	target changes when the live grid changes
Particle / TT lanes	richer non-Gaussian object	samples or density representation	different target class	heavier approximation object, different contract

For any deterministic Gaussian-surrogate lane M, write its induced scalar as

$$\ell_T^{(M)}(\theta) = \sum_{t=1}^{T} \log \mathcal{N}\big(y_t; \bar{z}_t^{(M)}(\theta), S_t^{(M)}(\theta)\big). \tag{8.2}$$

and its moment engine as the operator

$$(\bar{z}_t^{(M)}, S_t^{(M)}, C_{xz,t}^{(M)}) = \mathcal{Q}_M\big(f_\theta, h_\theta, m_{t-1}, P_{t-1}, Q_\theta, R_\theta\big). \tag{8.3}$$

The comparison is then mathematical rather than rhetorical: nearby Gaussian filters differ by the operator $\mathcal{Q}_M$ that feeds the same Gaussian update algebra, while particle and tensor-density methods differ already at the level of the carried object.

For a deterministic point rule with nodes $\xi_r^{(M)}$ and weights $w_r^{(M)}$, that operator has the explicit form

$$\bar{z}_t^{(M)} \approx \sum_{r=1}^{M_M} w_r^{(M)}\, h_\theta\big(m_t^- + C_t^- \xi_r^{(M)}\big), \tag{8.4}$$

$$S_t^{(M)} \approx R_\theta + \sum_{r=1}^{M_M} w_r^{(M)} \Delta_{t,r}^{(M)} \Delta_{t,r}^{(M)\top}, \qquad \Delta_{t,r}^{(M)} := h_\theta(m_t^- + C_t^- \xi_r^{(M)}) - \bar{z}_t^{(M)}, \tag{8.5}$$

$$C_{xz,t}^{(M)} \approx C_t^- \left(\sum_{r=1}^{M_M} w_r^{(M)} \, \xi_r^{(M)} \Delta_{t,r}^{(M)\top} \right). \tag{8.6}$$

Thus the decisive question is not whether these methods are all "Gaussian," but what kind of moment engine they use and what that engine implies for the scalar and its derivative.

What disqualifies a lane for this note? A lane is not selected here if any one of the following fails:

(i) a declared deterministic approximate scalar objective,
(ii) an analytical derivative of that same scalar on a fixed smooth branch, (8.7)
(iii) a representation that avoids dense tensor-product growth in the intended regime.

This is not a theorem of universal method superiority. It is the local choice architecture of this report. Conditions (i)–(iii) are used here as hard vetoes: a method that fails one of them is not selected for this lane. Once those conditions are met, the later comparisons concern which eligible lane best realizes the report's strong preferences.

EKF and local linearization. The extended Kalman filter carries the same kind of Gaussian object (m_t, P_t), but its moment engine is not (8.4)–(8.6). Instead it substitutes local linear surrogates,

$$f_\theta(x) \approx f_\theta(m) + F_\theta(x - m), \qquad h_\theta(x) \approx h_\theta(m^-) + H_\theta(x - m^-), \tag{8.8}$$

so that the predictive and observation moments are inherited from Jacobian closure rather than from explicit nonlinear point evaluation. In particular, the moment operator becomes

$$\mathcal{Q}_{\mathrm{EKF}} \approx \big(h_\theta(m_t^-), \; H_\theta P_t^- H_\theta^\top + R_\theta, \; P_t^- H_\theta^\top \big), \tag{8.9}$$

which is computationally cheap but sacrifices the explicit nonlinear pointwise moment information preserved by (8.4)–(8.6). The present report does not reject EKF as incoherent; it rejects it because the selected lane insists on evaluating the nonlinear maps themselves under a declared cloud.

Low-order sigma-point and cubature lanes. UKF and CKF remain viable deterministic Gaussian alternatives because they also construct a scalar of the form (8.2). Their difference lies in the point design,

$$\mathcal{Q}_{\mathrm{low}} \sim \sum_{r=1}^{M_{\mathrm{low}}} \tilde{w}_r F(\tilde{\xi}_r), \qquad M_{\mathrm{low}} = O(n_x), \tag{8.10}$$

where the low point count is attractive but the deterministic rule remains a lower-order moment design. FixedSGQF instead chooses the sparse-grid rule

$$\mathcal{Q}_{\mathrm{SGQF}} \sim \sum_{r=1}^{M_{b,L}} w_r F(\xi^{(r)}), \qquad M_{b,L} = O\big(b^{L-1}\big) \quad \text{for fixed } L, \tag{8.11}$$

The mathematical tradeoff is therefore visible in two dimensions at once:

$$M_{\mathrm{low}} = O(n_x) \qquad \text{versus} \qquad M_{b,L} = O\big(b^{L-1}\big), \tag{8.12}$$

while the corresponding moment operators satisfy

$$\mathcal{Q}_{\text{low}} \neq \mathcal{Q}_{\text{SGQF}} \quad \text{except in special low-order cases where the same moments are matched.} \tag{8.13}$$

So the comparison is not "deterministic versus nondeterministic," but low-order low-point deterministic moment closure versus higher-structure sparse-grid deterministic moment closure. The selected lane is willing to pay more than $O(n_x)$ points in exchange for a richer deterministic moment structure while still avoiding the tensor-product explosion of dense GHQF. The decisive point for this note is not that low-order sigma-point rules are invalid, but that once the project asks for a deterministic Gaussian-surrogate lane with a visibly recorded cloud, merge, and branch identity, the sparse-grid construction carries more of the moment-engine burden in the object itself rather than in a smaller rule whose adequacy must be argued indirectly.

Standard and tensor-product GHQF. Dense Gaussian quadrature remains the cleanest low-dimensional reference, but it fails the present lane in the intended regime because the point count grows as

$$M_{\text{tensor}}(b, L) = s_L^b, \tag{8.14}$$

where s_L is the one-dimensional point count at the chosen level. By contrast, the sparse-grid cloud in this report has

$$M_{\text{SGQF}}(b, L) = M_{b,L} = O\big(b^{L-1}\big) \qquad \text{for fixed } L. \tag{8.15}$$

The note does not claim that sparse-grid quadrature is uniformly more accurate than dense GHQF. It claims something narrower: if the lane is defined by deterministic Gaussian-surrogate likelihood evaluation in a high-dimensional regime, then dense GHQF is too expensive to be the selected lane and should remain a comparator or small-dimension reference.

Live adaptive sparse-grid selection. Adaptive sparse-grid methods are still useful, but not as the selected same-scalar derivative lane. If the active index set changes with θ, then the scalar itself becomes

$$\ell_T^{\text{adap}}(\theta) = \sum_{t=1}^{T} \ell_t\big(\theta, \mathcal{I}_t(\theta)\big), \tag{8.16}$$

where $\mathcal{I}_t(\theta)$ is the parameter-dependent active index set. Its formal derivative is then only local to regions where the index set is constant:

$$\partial_i \ell_T^{\text{adap}}(\theta) = \sum_{t=1}^{T} \partial_i \ell_t\big(\theta, \mathcal{I}_t(\theta)\big) \quad \text{only on regions where } \mathcal{I}_t(\theta) \text{ is locally constant.} \tag{8.17}$$

That is a different target class from the fixed scalar of this note. The role of adaptation here is therefore upstream cloud design, not the selected downstream same-scalar likelihood lane.

Particle and tensor-density lanes. Particle and TT methods differ already at the level of the carried object. Particle filtering carries weighted samples; squared TT carries a richer non-Gaussian density approximation. In schematic form,

$$\text{particle lane: carried object } \{x_t^{(r)}, \omega_t^{(r)}\}_{r=1}^{N}, \qquad \text{TT lane: carried object } \widehat{p}_t^{\,\text{TT}}(x), \tag{8.18}$$

whereas FixedSGQF carries only (m_t, P_t) together with the surrogate scalar. These methods are not ruled out because they are weak. They are retained as complementary lanes because they preserve posterior structure that one Gaussian surrogate cannot preserve. The present note is therefore not selecting the method with the broadest posterior-shape fidelity; it is selecting the deterministic Gaussian-surrogate lane with the cleanest same-scalar likelihood-and-gradient contract.

Mathematical selection summary. The comparison therefore closes not with a slogan but with the following local selection statement:

$$\text{Selected lane} = \text{deterministic Gaussian-surrogate scalar+fixed-branch analytical derivative+sparse-grid moment e} \tag{8.19}$$

This equation is not a theorem of universal optimality. It is the summary of the local choice architecture built in this report. FixedSGQF is selected because, among nearby deterministic Gaussian candidates, it is the lane that jointly preserves: a declared deterministic scalar, an analytical derivative of that same scalar on a fixed branch, an explicit approximation boundary, and a cost story less explosive than dense GHQF while more stable than live adaptive-grid selection. That is the precise sense in which this note chooses FixedSGQF.

9 Conclusion

This report has addressed a specific scientific problem: high-dimensional nonlinear filtering in a regime where exact posterior propagation is not a practical computational object, but where a deterministic and analytically differentiable surrogate may still be scientifically valuable. The proposal advanced here is FixedSGQF, a lane that carries one Gaussian surrogate, approximates its required nonlinear moments by a fixed sparse-grid cloud, and differentiates the resulting deterministic same-scalar likelihood on a declared branch.

What the report has constructed is therefore precise. It has reconstructed the Gaussian-projection sparse-grid filtering value path in source order, made the saved branch explicit, given a branch-conditioned analytical gradient of the same scalar that the value routine evaluates, provided a compact worked numeric oracle, and written the value and gradient recursions in implementation order. A reader should now be able to identify the exact carried state, the exact accumulated scalar, the exact derivative contract, and the exact places where a branch veto occurs.

The report has also been explicit about what it does *not* claim. It does not claim exact nonlinear likelihood evaluation. It does not claim that sparse-grid exactness for selected Gaussian moments implies exact non-Gaussian posterior shape. It does not claim that one carried Gaussian can preserve multimodality, heavy asymmetry, or all-coordinate interaction structure whenever those features matter scientifically. It does not replace richer non-Gaussian density-approximation lanes such as Zhao–Cui squared TT.

The strongest scientific case for FixedSGQF is therefore not generic novelty but a specific combination of properties. The cloud is declared, the covariance-factor convention is declared, the branch-veto logic is declared, the scalar is deterministic on that branch, and the analytical gradient differentiates that same declared scalar. Those properties are exactly the criteria the report used to distinguish FixedSGQF from its nearest deterministic Gaussian alternatives: more explicit nonlinear point evaluation than EKF, a more transparent sparse-grid cloud contract than UKF/CKF, a less explosive cost story than dense GHQF, and a cleaner same-scalar derivative contract than live adaptive grid selection. This makes the method well suited to settings where reproducibility, inspectability, and gradient-based parameter learning matter, and where the filtering task can tolerate a Gaussian-surrogate carried object. In regimes with lower effective interaction order, the sparse-grid moment engine can also be scientifically plausible without requiring a full tensor-product explosion.

The main caution is equally clear. FixedSGQF should not be oversold as a full non-Gaussian posterior method. When posterior shape itself is central — for example under strong multimodality, hard global interactions, or branch fragility — the deterministic Gaussian-surrogate lane becomes

scientifically too narrow, and richer sample-based or density-based methods deserve priority. That is not a defect in the present report; it is the explicit boundary that keeps the proposal honest.

A fair review-panel judgment should therefore be the following. FixedSGQF is a coherent, mathematically explicit, self-contained, and mathematically specified for implementation and audit high-dimensional filtering lane. It is narrower than a full non-Gaussian density approximation, but that narrowness buys a transparent deterministic surrogate and a branch-conditioned analytical gradient that can be audited, implemented, and checked directly. The approval claim remains bounded: this report is not asserting that FixedSGQF is the best nonlinear filter overall, but that it is the selected deterministic Gaussian-surrogate lane for the explicit approximate likelihood-and-gradient objective developed here. On that basis, this report provides a coherent and auditable basis for further implementation and validation review of FixedSGQF as one serious lane in a broader high-dimensional filtering program.

A Implementation Contract

object	definition	shape	reuse
$\xi^{(r)}$	standardized sparse-grid node	n_x	value and gradient
w_r	accumulated signed weight after duplicate merge	scalar	value and gradient
C_{t-1}, C_t^-	saved covariance factors on declared branch	$n_x \times n_x$	value, gradient, diagnostics
$\chi_{t-1}^{(r)}$	previous placed point	n_x	transition value and derivative
$a_t^{(r)}$	transition image	n_x	prediction moments
$\chi_t^{(r)}$	predictive placed point	n_x	observation value and derivative
$z_t^{(r)}$	observation image	n_y	observation moments
$\bar{z}_t, S_t, C_{xz,t}$	Gaussian projection moments	n_y, $n_y \times n_y$, $n_x \times n_y$	likelihood and update
v_t, K_t, u_t	innovation, Gaussian projection gain, and innovation solve variable with $S_t u_t = v_t$	n_y, $n_x \times n_y$, n_y	likelihood, update, gradient
$\dot{\chi}, \dot{a}, \dot{z}$	point sensitivities	matching value objects	gradient only
$\dot{m}, \dot{P}, \dot{S}, \dot{C}_{xz}$	moment and filter sensitivities	matching value objects	gradient and next step
data record	$y_{1:T}$, preprocessing, missing-data policy if any	metadata	value, gradient, finite difference
initial law	$m_0(\theta), P_0(\theta), \dot{m}_0, \dot{P}_0$	$n_x, n_x \times n_x$	value and gradient start
branch record	cloud, one-dimensional rules, merge tolerance, zero-weight rule, node ordering, preprocessing policy, initial-condition policy, declared Cholesky factor branch, symmetrize-then-veto rule, thresholds, accepted/failure stage-time pattern	metadata	same-scalar finite difference

A.1 Input, Output, Invariant, And Failure Contract

The full report-level routine has inputs

$$\begin{aligned}\mathcal{I}_{\mathrm{in}} = (&y_{1:T}, \theta, L, \{I_\ell\}_{\ell=1}^{L+b-1}, m_0(\theta), P_0(\theta),\\ &\dot{m}_0^{(1:p)}(\theta), \dot{P}_0^{(1:p)}(\theta), f_\theta, h_\theta, Q_\theta, R_\theta,\\ &D_x f_\theta, \partial_{1:p} f_\theta, D_x h_\theta, \partial_{1:p} h_\theta, \dot{Q}_\theta^{(1:p)}, \dot{R}_\theta^{(1:p)},\\ &\text{merge tolerance, factor map, branch thresholds}).\end{aligned} \tag{A.1}$$

It returns either

$$\left(\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta), \nabla_\theta \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta), \{m_t, P_t\}_{t=0}^T, \mathcal{D}_{\mathrm{diag}}\right) \tag{A.2}$$

or a failure record

$$\mathcal{F} = (\text{time, stage, reason, diagnostic values}). \tag{A.3}$$

The invariants are:

$$\sum_{r=1}^{M} w_r = 1 \qquad \text{up to construction tolerance,} \tag{A.4}$$

$$P_t^- = (P_t^-)^\top, \quad S_t = S_t^\top, \quad P_t = P_t^\top \qquad \text{after explicit symmetrization,} \tag{A.5}$$

$$S_t \succ 0 \qquad \text{on every accepted likelihood step,} \tag{A.6}$$

$$\mathcal{B}_{\mathrm{value}} = \mathcal{B}_{\mathrm{gradient}} \qquad \text{same saved branch.} \tag{A.7}$$

The routine fails rather than repairs if

$$\lambda_{\min}(S_t) < \epsilon_S, \qquad \lambda_{\min}(P_t^-) < \epsilon_P, \qquad \lambda_{\min}(P_t) < \epsilon_P, \tag{A.8}$$

or if the declared factor map cannot be evaluated on the symmetrized covariance. An implementation may add a smoothed repair, but then the repair is a new scalar and needs a new derivative.

A.2 One-Step Value-Path Contract

At one accepted time step, the value path is the ordered map

$$\begin{aligned}(m_{t-1}, P_{t-1}, y_t, \theta, \mathcal{C}) &\longmapsto C_{t-1} \longmapsto \chi_{t-1}^{(r)} \longmapsto a_t^{(r)} \longmapsto (m_t^-, P_t^-)\\ &\longmapsto C_t^- \longmapsto \chi_t^{(r)} \longmapsto z_t^{(r)} \longmapsto (\bar{z}_t, S_t, C_{xz,t})\\ &\longmapsto (v_t, \widehat{\ell}_t^{\mathrm{FSGQ}}, K_t) \longmapsto (m_t, P_t).\end{aligned} \tag{A.9}$$

The accepted map in (A.9) is conditioned on two successful branch checks: the predictive covariance P_t^- must pass the declared Cholesky and positive-definite test before observation-point placement, and the innovation covariance S_t must pass the declared branch test before the Gaussian projection update is accepted. The accepted outputs propagated to the next step are only (m_t, P_t) together with the global fixed branch record $\mathcal{B}$. Point clouds, moment arrays, gains, and diagnostic quantities may be cached for gradient reuse, but they are not part of the carried filtering state itself.

A.3 Default Numerical Constants

The constants below are defaults for diagnostics, not claims of universal validity:

quantity	default	role
duplicate merge tolerance	$10^{-12}\max(1, \|\xi\|_\infty)$	groups standardized nodes that should be identical under the declared rule
weight zero tolerance	10^{-14}	removes accumulated numerical-zero weights after merge
ϵ_S	10^{-10}	veto threshold for innovation covariance positive definiteness
ϵ_P	10^{-10}	veto threshold for carried and predictive covariance factors
ϵ_ℓ	$10^{-3}\max(1, \|\widehat{\ell}_T\|)$	pilot level-ladder change threshold
finite-difference relative tolerance	10^{-5}	same-scalar gradient diagnostic
maximum branch-invalid FD rows	0 in final audit	branch changes mean the derivative is not being tested

These values are deliberately conservative. They are meant to reveal when the fixed branch is fragile. They are not tuning knobs for making a failing branch look smooth.

B End-To-End Mathematical Algorithm

FixedSGQF value and gradient, with all branch exits.

1. Build the sparse-grid cloud from $b = n_x$, level L, and $\{I_\ell\}$. Use (3.47), (3.49), (3.64), (3.65), and (3.67). Save $\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M}$.
2. Check (A.4). If it fails, return $\mathcal{F} = (0, \text{cloud}, \text{weight-sum failure}, \cdots)$.
3. Initialize $\widehat{\ell}_T^{\mathrm{FSGQ}} \leftarrow 0$, $G_i \leftarrow 0$ for $i = 1, \ldots, p$, and compute $m_0, P_0, \dot{m}_0^{(i)}, \dot{P}_0^{(i)}$.
4. For $t = 1, \ldots, T$, symmetrize P_{t-1}. If the factor map $C(P_{t-1})$ fails, return a factor-branch failure. Otherwise set $C_{t-1} = C(P_{t-1})$ and for each parameter component compute $\dot{C}_{t-1}^{(i)} = DC(P_{t-1})[\dot{P}_{t-1}^{(i)}]$.
5. Place previous points by (4.2); compute their derivatives by (5.14) componentwise in $i = 1, \ldots, p$.
6. Evaluate transition values (4.3) and derivatives (5.15).
7. Compute m_t^-, P_t^-, $\dot{m}_t^{-(i)}$, and $\dot{P}_t^{-(i)}$ by (4.4), (4.5), (5.16), and (5.18). Symmetrize P_t^-. If (A.8) fails for P_t^-, return a predictive covariance failure.
8. Factor P_t^-. Set $C_t^- = C(P_t^-)$ and compute $\dot{C}_t^{-(i)} = DC(P_t^-)[\dot{P}_t^{-(i)}]$ for each parameter component. Place observation points by (4.6); compute their derivatives by (5.20).
9. Evaluate observation values (4.7) and derivatives (5.21).
10. Compute $\bar{z}_t, \Delta_t^{(r)}, S_t, C_{xz,t}, v_t$ by (4.8)–(4.12). Symmetrize S_t. If S_t fails (A.8), return an innovation covariance failure.
11. Only on that accepted innovation branch, form $\widehat{\ell}_t^{\mathrm{FSGQ}}$ by (4.13).
12. Compute $\dot{\bar{z}}_t^{(i)}, \dot{v}_t^{(i)}, \dot{S}_t^{(i)}$, and $\dot{C}_{xz,t}^{(i)}$ by (5.22)–(5.26).
13. Solve $S_t u_t = v_t$. Add $\widehat{\ell}_t^{\mathrm{FSGQ}}$ to $\widehat{\ell}_T^{\mathrm{FSGQ}}$ only after the innovation branch has passed, and add the componentwise score to G_i only on that same accepted innovation branch. Reuse this same solved u_t for every parameter component i, and add the componentwise score from (5.28) with $\dot{v}_t = \dot{v}_t^{(i)}$ and $\dot{S}_t = \dot{S}_t^{(i)}$ to G_i.
14. Compute K_t first from (4.14). Then for each parameter component compute $\dot{K}_t^{(i)}$ from (5.33), and update $m_t, P_t, \dot{m}_t^{(i)}, \dot{P}_t^{(i)}$ by (4.15)–(4.16) and (5.34)–(5.35). Symmetrize P_t. If the carried covariance branch fails, return a carried covariance failure.
15. Return $(\widehat{\ell}_T^{\mathrm{FSGQ}}, G_{1:p}, \{m_t, P_t\}, \mathcal{D}_{\mathrm{diag}})$.

The algorithm is deliberately mathematical rather than Pythonic. Its purpose is to say what a Python implementation must compute, save, and refuse to change if it wants the reported gradient to be the derivative of the reported value.

B.1 Formula Inventory For The Value Path

For implementation review, the value path can be located in the report as follows.

value-path item	report location
symbol and dimension table	Section 2.1
exact filtering target	Section 2
Gaussian projection moment equations	Section 2.1.1
standardized sparse-grid cloud construction	Section 3.4.1
merge convention and stored cloud semantics	Section 3.4.6 and Section 3.4.9
per-step value inputs and outputs	Sections 4, A.1, and A.2
prediction equations	(4.2)–(4.5)
observation and update equations	(4.6)–(4.16)
one-step log-likelihood increment	(4.13)
implementation-order value/gradient algorithm	Section B
branch checks and numerical conventions	Sections 2.1, A.1, and A.3

C Where Each Construction Comes From

construction	support	use in this note
state-space and conceptual filtering equations	Jia et al. [1, Section 2]	Sections 2–2.1.1
Gaussian approximation filter moment structure	Jia et al. [1, Eq. 15–23] plus Proposition 1	moment projection and update equations
one-dimensional and tensor-product Gaussian quadrature, with the present note fixing a GHQ-specialized odd-point rule family	Jia et al. [1, Section 2.2.1 and Eq. 34–35]	Section 3.3
sparse-grid formula and index band	Jia et al. [1, Eq. 26–29]	Section 3.4.1
point/weight generation and duplicate accumulation	Jia et al. [1, Algorithm 1]	fixed cloud $\mathcal{C}$ and toy grid
quadrature exactness and UKF relation	Jia et al. [1, Theorems 3.1–3.2]	source-scope orientation, not posterior guarantee
adaptive sparse-grid mechanics	Singh et al. [2, Section 3]	offline grid-design interpretation
fixed scalar and analytical gradient	BayesFilter project derivation	Sections 5–5.0.1
comparison with squared TT	Zhao–Cui companion note and Zhao and Cui [3]	Section 8

D Notation And Formula Inventory

For implementation review, the note can be navigated by the following inventory.

item	report location
scientific problem and lane choice	Sections 1.1–1.2
exact object versus approximate object	(1.2), Section 1.4
global notation and dimensions	Section 2.1
Gaussian projection equations	Section 2.1.1
source-order sparse-grid construction	Section 3.4.1
merge convention and stored cloud semantics	Section 3.4.6, Section 3.4.9
value-path recursion	Section 4, Section A.2, Section B.1
fixed scalar and saved-branch contract	Section 5, (5.2), (5.4)
factor derivative convention	Section 5.0.1, subsection ‘Square-Root Branch’, equations (5.10)–(5.12)
gradient dependency chain and roles	(5.5), (5.8), (5.9)
prediction and observation sensitivities	(5.14)–(5.26)
innovation-score derivative	Proposition 2, (5.28)
posterior sensitivity propagation	(5.32), (5.33)–(5.35)
one-component summary algorithm	Section 5.1
full implementation-order algorithm	Section B
worked numeric oracle	Section 4.3
finite-difference same-scalar diagnostics	Section 6
neighboring-method positioning	Section 8
validation models and report template	Section 6.0.1, Section 6.0.3

References

[1] Bin Jia, Ming Xin, and Yang Cheng. Sparse-grid quadrature nonlinear filtering. *Automatica*, 48 (2):327–341, 2012. doi: 10.1016/j.automatica.2011.08.057.

[2] Abhinoy Kumar Singh, Rahul Radhakrishnan, Shovan Bhaumik, and Paresh Date. Adaptive sparse-grid gauss-hermite filter. *arXiv preprint arXiv:1803.09272*, 2018.

[3] Yiran Zhao and Tiangang Cui. Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25:1–51, 2024.